A

MINI PROJECT REPORT ON

“SMART COLLEGE FAQ SYSTEM”

SUBMITTED TOWARDS THE PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE AWARD OF THE DEGREE OF

[BACHELOR OF TECHNOLOGY]

IN

INFORMATION TECHNOLOGY

By

ROLL NO.	PRN NO.	NAME
61	UIT23M1061	Onkar Anil Gunjal
62	UIT23M1062	Chaitanya Sudhakar Gurav
63	UIT23M1063	Ritesh Bapusaheb Gursal
64	UIT23M1064	Abhijit Sachin Hajare

Under the Guidance of

Dr. Sachin Thanekar

Department of Information Technology
Sanjivani College of Engineering, Kopargaon-423603

Academic Year 2026–2027

CERTIFICATE

This is to certify that the mini project report entitled “Smart College FAQ System” is a bonafide record of work carried out by:

ROLL NO.	PRN NO.	NAME
61	UIT23M1061	Onkar Anil Gunjal
62	UIT23M1062	Chaitanya Sudhakar Gurav
63	UIT23M1063	Ritesh Bapusaheb Gursal
64	UIT23M1064	Abhijit Sachin Hajare

in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Information Technology of Sanjivani College of Engineering, Kopargaon during the academic year 2026–27, under my/our supervision.

Project Guide
Dr. Sachin Thanekar

Head of Department
Dr. Madhuri Jawale

Internal Examiner: _____

External Examiner: _____

Date:

Place: Kopargaon

DECLARATION

We hereby declare that the mini project report titled “Smart College FAQ System”, submitted in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Information Technology, is an original and independent piece of work carried out by us under the guidance of Dr. Sachin Thanekar.

We further declare that this report has not been submitted, in part or in full, to this or any other Institute/University for the award of any other degree or diploma. All sources of information, software libraries and literature used in this report have been duly acknowledged in the References section.

ROLL NO.	PRN NO.	NAME
61	UIT23M1061	Onkar Anil Gunjal
62	UIT23M1062	Chaitanya Sudhakar Gurav
63	UIT23M1063	Ritesh Bapusaheb Gursal
64	UIT23M1064	Abhijit Sachin Hajare

Date:
Place: Kopargaon

APPROVAL / ACCEPTANCE SHEET

The mini project report entitled “Smart College FAQ System”, submitted by Onkar Anil Gunjal, Chaitanya Sudhakar Gurav, Ritesh Bapusaheb Gursal and Abhijit Sachin Hajare, is hereby approved as satisfying the requirements for the award of the degree of Bachelor of Technology in Information Technology.

Examiner 1: _____ Examiner 2: _____

Date: Place: Kopargaon

ACKNOWLEDGEMENT

We take this opportunity to express our sincere gratitude to our project guide, Dr. Sachin Thanekar, for his valuable guidance, continuous encouragement and constructive suggestions throughout the development of this project, “Smart College FAQ System.” His guidance in the area of Natural Language Processing helped us shape both the design and the evaluation of this work.

We are thankful to Dr. Madhuri Jawale, Head of the Department of Information Technology, and to all the faculty members of the department for their support and for providing the facilities required to carry out this project.

We also extend our thanks to the Principal and the management of Sanjivani College of Engineering, Kopargaon for providing an environment that encourages practical, project-based learning.

Finally, we thank our families, classmates and everyone who directly or indirectly supported and motivated us during the completion of this project.

ROLL NO.	PRN NO.	NAME
61	UIT23M1061	Onkar Anil Gunjal
62	UIT23M1062	Chaitanya Sudhakar Gurav
63	UIT23M1063	Ritesh Bapusaheb Gursal
64	UIT23M1064	Abhijit Sachin Hajare

ABSTRACT

Students regularly need answers to routine questions about admissions, examinations, fees, the library, hostel accommodation, attendance, placements and scholarships. In many colleges this information is spread across notices and departments, and the same questions are answered repeatedly by staff. Keyword-based search tools help only when the student uses the same words as the stored information. This project presents the design and implementation of the Smart College FAQ System, a Natural Language Processing (NLP) based question answering application that accepts a question written in ordinary English and retrieves the most relevant entry from a predefined college FAQ dataset.

The system follows a retrieval approach rather than generating new text. The FAQ dataset (data/faq_data.csv) contains 34 question–answer records grouped into 11 categories. Each question is preprocessed by lowercasing, removing non-alphabetic characters, tokenizing, removing English stopwords with NLTK and lemmatizing with the WordNet lemmatizer. Two similarity signals are then computed against every FAQ: a lexical score from TF-IDF vectors and cosine similarity (scikit-learn), and a semantic score from 384-dimensional sentence embeddings produced by the pretrained Sentence Transformer model all-MiniLM-L6-v2. The hybrid score is calculated as 0.30 × TF-IDF similarity + 0.70 × semantic similarity, with a small boost of 0.05 when an important intent word appears in both questions, and is clipped to the range 0 to 1. If the best score is below a threshold of 0.50, the system returns a fallback message and the category "Unknown" instead of a misleading answer.

The retrieval engine is exposed through a FastAPI backend with endpoints for health checking, question answering, listing FAQs, listing categories and viewing question history. Every answered question is stored in an SQLite database. A Streamlit web interface allows students to type a question or choose a popular question and displays the answer together with its category, match confidence and the matched FAQ. On the project's 14-case API validation set, consisting of 11 college-related questions and 3 unrelated questions, the system produced the expected category for all 14 cases (100% accuracy on this validation set). The college questions received confidence values between 82.33% and 100.00%, while the unrelated questions scored between 10.33% and 15.85% and were correctly rejected. These results are limited to the available test set and do not represent accuracy on all possible student questions.

KEYWORDS

Natural Language Processing (NLP); FAQ Retrieval; Question Answering; Text Preprocessing; Stopword Removal; Lemmatization; TF-IDF; Cosine Similarity; Sentence Transformers; all-MiniLM-L6-v2; Semantic Similarity; Hybrid Similarity; Threshold-Based Retrieval; FastAPI; Streamlit; SQLite; College Information System.

TABLE OF CONTENTS

Certificate.....**i**
Declaration.....**ii**
Approval / Acceptance Sheet.....**iii**
Acknowledgement.....**iv**
Abstract.....**v**
Keywords.....**v**
List of Figures.....**ix**
List of Tables.....**x**
List of Abbreviations.....**xi**
CHAPTER 1 INTRODUCTION.....**1**
1.1 Background.....1
1.2 Introduction to Smart College FAQ System.....1
1.3 Need for the System.....1
1.4 Motivation.....1
1.5 Problem Statement.....1
1.6 Objectives.....2
1.7 Scope of the Project.....2
1.8 Advantages.....2
1.9 Limitations.....2
1.10 Organization of the Report.....2
CHAPTER 2 PROBLEM IDENTIFICATION AND ANALYSIS.....**3**
2.1 Existing Problem.....3
2.2 Problems with Traditional College FAQ Handling.....3
2.3 Challenges Faced by Students.....3
2.4 Existing System.....3
2.5 Limitations of Existing Approach.....3
2.6 Proposed System.....4
2.7 Proposed Solution.....4
2.8 Comparison Between Existing and Proposed System.....4
CHAPTER 3 REQUIREMENT GATHERING AND ANALYSIS.....**5**
3.1 Requirement Gathering.....5
3.2 Functional Requirements.....5
3.3 Non-Functional Requirements.....5
3.4 Hardware Requirements.....6
3.5 Software Requirements.....6
3.6 User Requirements.....7
3.7 System Constraints.....7
3.8 Feasibility Analysis.....7
CHAPTER 4 SYSTEM DESIGN AND ARCHITECTURE.....**8**
4.1 System Overview.....8
4.2 Overall Architecture.....8
4.3 System Components.....9
4.4 Data Flow.....9
4.5 NLP Processing Pipeline.....10
4.6 Query Processing.....11
4.7 FAQ Retrieval Process.....12
4.8 Hybrid Similarity Architecture.....12
4.9 Backend Architecture.....13
4.10 Frontend Architecture.....13
4.11 Database Architecture.....14
4.12 API Architecture.....14

CHAPTER 5 TECHNOLOGY STACK.....15
5.1 Python.....15
5.2 Streamlit.....15
5.3 FastAPI.....15
5.4 Natural Language Processing.....15
5.5 NLTK.....15
5.6 TF-IDF.....15
5.7 Cosine Similarity.....15
5.8 Sentence Transformers.....16
5.9 all-MiniLM-L6-v2.....16
5.10 Pandas.....16
5.11 Scikit-learn.....16
5.12 SQLite.....16
5.13 Requests.....16
5.14 Development Environment.....16
5.15 VS Code.....16
CHAPTER 6 DATASET AND NLP PROCESSING.....17
6.1 Dataset Overview.....17
6.2 FAQ Dataset Structure.....17
6.3 Dataset Fields.....17
6.4 FAQ Categories.....17
6.5 Data Preprocessing.....18
6.6 Lowercasing.....19
6.7 Special Character Removal.....19
6.8 Tokenization.....19
6.9 Stopword Removal.....19
6.10 Lemmatization.....19
6.11 TF-IDF.....19
6.12 Cosine Similarity.....20
6.13 Semantic Embeddings.....20
6.14 Sentence Transformer.....20
6.15 Hybrid Similarity.....20
6.16 Threshold-Based Retrieval.....21
6.17 Unknown Question Handling.....21
CHAPTER 7 SYSTEM IMPLEMENTATION.....22
7.1 Implementation Overview.....22
7.2 Project Structure.....22
7.3 Preprocessing Implementation.....22
7.4 FAQ Engine Implementation.....23
7.5 Hybrid Engine Implementation.....23
7.6 Semantic Model Integration.....24
7.7 FastAPI Backend Implementation.....24
7.8 API Endpoint Implementation.....24
7.9 Database Implementation.....25
7.10 Streamlit Frontend Implementation.....25
7.11 Backend-Frontend Communication.....25
7.12 Error Handling.....26
7.13 Unknown Query Handling.....26
CHAPTER 8 USER INTERFACE DESIGN.....27
8.1 UI Overview.....27
8.2 Home Screen.....27
8.3 Question Input.....28
8.4 Popular Questions.....28
8.5 Answer Display.....28

8.6 Category Display....30
8.7 Confidence Display....30
8.8 Matched FAQ Display....30
8.9 Unknown Question Response....30
8.10 User Experience....31
CHAPTER 9 API AND BACKEND....32
9.1 Backend Overview....32
9.2 FastAPI....32
9.3 API Endpoint....32
9.4 Request Structure....32
9.5 Response Structure....32
9.6 Swagger Documentation....33
9.7 Backend Processing Flow....34
9.8 Database/History....34
9.9 Error Handling....34
CHAPTER 10 TESTING AND VALIDATION....36
10.1 Testing Strategy....36
10.2 Unit Testing....36
10.3 Preprocessing Testing....36
10.4 FAQ Retrieval Testing....36
10.5 Semantic Testing....37
10.6 Hybrid Testing....37
10.7 API Testing....37
10.8 Positive Test Cases....37
10.9 Negative Test Cases....38
10.10 Unknown Question Testing....38
10.11 Test Results....39
10.12 Accuracy Calculation....39
CHAPTER 11 RESULTS AND DISCUSSION....40
11.1 System Results....40
11.2 FAQ Retrieval Performance....40
11.3 Semantic Matching Results....41
11.4 Hybrid Matching....42
11.5 Unknown Query Detection....42
11.6 API Performance....42
11.7 UI Evaluation....42
11.8 Discussion....42
11.9 Observed Limitations....42
CHAPTER 12 LIMITATIONS AND FUTURE SCOPE....43
12.1 Current Limitations....43
12.2 Dataset Limitations....43
12.3 NLP Limitations....43
12.4 Dependency on FAQ Dataset....43
12.5 Possible Improvements....43
12.6 Larger FAQ Dataset....44
12.7 Multilingual Support....44
12.8 Voice-Based Queries....44
12.9 Admin FAQ Management....44
12.10 Improved Ranking....44
12.11 Authentication....44
12.12 Cloud Deployment....44
12.13 Advanced Conversational Capabilities....44
CHAPTER 13 CONCLUSION....45
13.1 Summary of the Work....45

13.2 NLP Approach and Hybrid Retrieval..... 45
13.3 Backend, Frontend and Data Storage..... 45
13.4 Testing and Results..... 45
13.5 Practical Usefulness and Future Improvement..... 45
REFERENCES..... 46
APPENDIX A PROJECT STRUCTURE..... 47
APPENDIX B DATASET SAMPLE..... 48
APPENDIX C IMPORTANT SOURCE CODE..... 49
C.1 preprocessing.py..... 49
APPENDIX D API EXAMPLES..... 50
APPENDIX E TEST RESULTS..... 51
APPENDIX F ADDITIONAL SCREENSHOTS..... 52

LIST OF FIGURES

Figure No.	Title	Page No.
4.1	Use Case Diagram of the Smart College FAQ System	8
4.2	Overall System Architecture	9
4.3	Context-Level Data Flow Diagram (Level 0)	10
4.4	Level-1 Data Flow Diagram	10
4.5	NLP Processing Pipeline	11
4.6	Query Processing Flowchart	12
4.7	Hybrid Similarity Architecture	13
4.8	Backend Initialization Sequence	13
4.9	Frontend–Backend Communication Architecture	14
4.10	Database Schema and Access Operations	14
6.1	Distribution of FAQ Records Across Categories	18
6.2	Preprocessing Steps Applied to a Sample Question	18
7.1	Module Dependency and Testing Workflow	22
8.1	Smart College FAQ System Home Interface	28
8.2	Library FAQ Response	29
8.3	Hostel FAQ Response	29
8.4	Placement FAQ Response	30
8.5	Unknown Query Handling in the User Interface	31
9.1	FastAPI Swagger Documentation Interface	33
9.2	API Response for an Unknown Question in Swagger UI	34
9.3	Sequence Diagram for Processing a Question Through POST /ask	34
10.1	Hybrid Confidence for the 14 API Validation Cases	39
11.1	TF-IDF Baseline Score Compared with Hybrid Confidence	41
F.1	test_api.py Executed While the Backend Was Not Running (VS Code)	52

LIST OF TABLES

Table No.	Title	Page No.
2.1	Comparison Between Existing Approach and Proposed System	4
3.1	Functional Requirements	5
3.2	Non-Functional Requirements	6
3.3	Hardware Requirements	6
3.4	Software Requirements	7
4.1	System Components and Their Responsibilities	9
6.1	Dataset Fields	17
6.2	FAQ Categories and Record Counts	18
6.3	Preprocessing Outputs from test_preprocessing.py	19
6.4	Intent Words Used for the Score Boost	20
6.5	Retrieval Parameters Used in the Project	21
7.1	Project File Structure	22
8.1	User Interface Components	27
9.1	API Endpoints and Responses	32
9.2	Response Fields of POST /ask	32
10.1	Test Scripts and Their Purpose	36
10.2	TF-IDF Engine Test Results (test_engine.py)	37
10.3	Positive API Test Cases	38
10.4	Negative API Test Cases	38
10.5	API Validation Test Summary	39
11.1	TF-IDF Baseline Score and Hybrid Confidence for the Validation Questions	40
11.2	Decomposition of Hybrid Scores	41
12.1	Summary of Current Limitations	43
12.2	Summary of Future Scope	44
B.1	FAQ Questions in data/faq_data.csv	48
E.1	Stored Records of the API Validation Run (Records 29 to 42)	51

LIST OF ABBREVIATIONS

Abbreviation	Full Form
NLP	Natural Language Processing
FAQ	Frequently Asked Questions
TF-IDF	Term Frequency – Inverse Document Frequency
TF	Term Frequency
IDF	Inverse Document Frequency
SBERT	Sentence-BERT (Sentence Bidirectional Encoder Representations from Transformers)
BERT	Bidirectional Encoder Representations from Transformers
API	Application Programming Interface
REST	Representational State Transfer
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
UI	User Interface
UX	User Experience
CSV	Comma-Separated Values
SQL	Structured Query Language
DB	Database
NLTK	Natural Language Toolkit
ASGI	Asynchronous Server Gateway Interface
OAS	OpenAPI Specification
DFD	Data Flow Diagram
ER	Entity–Relationship
CPU	Central Processing Unit
GPU	Graphics Processing Unit
RAM	Random Access Memory
IDE	Integrated Development Environment
CSS	Cascading Style Sheets
HTML	HyperText Markup Language
URL	Uniform Resource Locator

CHAPTER 1 INTRODUCTION

1.1 Background

Every academic year, students ask a large number of similar questions about admissions, examinations, fee payment, library rules, hostel accommodation, attendance, placements and scholarships. In most colleges this information is spread across notices, department offices and websites, and students often have to visit an office or ask a faculty member simply to find out where a piece of information is available. Static Frequently Asked Questions (FAQ) pages reduce this repetition only partly: the student must browse the list and guess which entry matches the question, and the matching becomes difficult when the student uses different words. A student who asks "Where do I get books?" may not recognise that the answer is stored under "Where can I get books from the library?".

Natural Language Processing (NLP) offers a practical way to close this gap. An NLP-based system can accept a question typed in ordinary English, compare it with all stored FAQ questions and return the answer of the closest match.

1.2 Introduction to Smart College FAQ System

The Smart College FAQ System is a web-based question answering application. The student types a question such as "How can I get a hostel room?" and the system retrieves the most relevant answer from a predefined college FAQ dataset. The system does not generate new text; every answer shown comes directly from the dataset, which keeps the answers controlled and predictable.

Internally, the system combines lexical TF-IDF (Term Frequency–Inverse Document Frequency) similarity with semantic similarity from the pretrained model all-MiniLM-L6-v2, and reports when no relevant answer exists instead of forcing one. The project consists of the NLP engine (`preprocessing.py`, `hybrid_engine.py`), a FastAPI backend (`backend/main.py`) with SQLite history storage (`backend/database.py`), and a Streamlit frontend (`app.py`).

1.3 Need for the System

Students need one place where common questions can be asked at any time, in their own words rather than the official wording of notices. A system that clearly reports when it cannot answer is safer than one that always returns something, because a wrong answer about fees or examinations can mislead a student. Recording the questions asked also shows which topics are missing from the dataset.

1.4 Motivation

Paraphrasing is natural in everyday language, yet keyword matching treats paraphrases as only partly related. Pretrained models such as all-MiniLM-L6-v2 make semantic matching feasible without training data, and combining them with the transparent TF-IDF technique allows a practical study of how lexical and semantic signals complement each other. FastAPI and Streamlit

make it possible to build a complete system with an API and an interface within a mini project timeline.

1.5 Problem Statement

Students frequently need answers to routine college-related questions, but this information is not available through a single interface that understands natural language. Static FAQ lists require manual browsing, and keyword search fails when the student's wording differs from the stored text. The problem addressed in this project is to design and implement a system that accepts a free-text question, identifies the most relevant entry in a predefined college FAQ dataset using both lexical and semantic similarity, returns the answer with its category and a confidence value, and rejects unrelated questions instead of returning a misleading answer.

1.6 Objectives

- To prepare a structured college FAQ dataset with a question, answer and category for each record.
- To implement text preprocessing: lowercasing, special character removal, tokenization, stopword removal and lemmatization.
- To implement a TF-IDF baseline engine and integrate all-MiniLM-L6-v2 for semantic similarity.
- To combine both scores into a hybrid score with an intent-word boost and a confidence threshold.
- To expose the engine through a FastAPI backend and store question history in SQLite.
- To develop a Streamlit interface that displays the answer, category, confidence and matched FAQ.
- To validate the system with script-based tests at the preprocessing, retrieval and API levels.

1.7 Scope of the Project

The scope covers the pipeline from a typed question to a displayed answer for the eleven categories in the dataset: Admission, Examination, Fees, Library, Hostel, Attendance, Placement, Scholarship, Academic, Student Activities and General. The system supports English questions and runs locally, with the Streamlit frontend calling the FastAPI backend at `http://127.0.0.1:8000`. Login, administrator screens, multilingual or voice input and cloud deployment are outside the scope (Chapter 12). The dataset answers are sample answers, not official college policies.

1.8 Advantages

Students can ask questions in their own words; answers always come from the dataset; unrelated questions are rejected with a clear message; and the confidence and matched FAQ make every answer transparent, while history is stored automatically.

1.9 Limitations

Only the 34 dataset records (with sample answers) can be used, only English is supported, the weights, boost and threshold were set manually, and the 14-case validation set does not allow results to be generalised to all student questions.

1.10 Organization of the Report

Chapters 2 and 3 cover problem analysis and requirements; Chapters 4 to 6 present the design, technologies and NLP methods; Chapters 7 to 9 describe the implementation, interface and API; Chapters 10 and 11 present testing and results; and Chapters 12 and 13 give the future scope and conclusion.

CHAPTER 2 PROBLEM IDENTIFICATION AND ANALYSIS

2.1 Existing Problem

Students think in terms of a need, such as "I need a hostel room", whereas institutional information is organised by office, notice or document. Bridging this gap usually requires a person: a clerk, a faculty member or a senior student. A second aspect of the problem is the variety of wording: "Where can I find exam notices?", "Where can I see examination information?" and "When will the exam schedule be available?" all concern examination information. Any system that depends on particular keywords will fail for some of these phrasings, so the problem is not only storing answers but matching a free-text question to the correct stored answer.

2.2 Problems with Traditional College FAQ Handling

- **Notice boards and circulars** are difficult to search later; a student who missed a notice must locate it physically.
- **Static FAQ pages** require the student to browse and compare entries manually.
- **Keyword search** ranks entries by shared words; if the student writes "books" while the entry says "library", the relevant entry may not rank first.
- **Office enquiries** depend on office hours, and the same questions are answered repeatedly.
- **Informal sources** such as classmates and messaging groups may provide outdated information.

2.3 Challenges Faced by Students

- Not knowing which office handles a topic or which official terms are used in notices.
- Spending time visiting offices or waiting for replies to simple questions.
- Receiving different answers from different informal sources.
- Hesitating to ask basic questions repeatedly, especially as a new student.

2.4 Existing System

In the existing approach, FAQ information is provided manually by staff or through static digital content, with keyword matching where a search facility exists. This keyword approach is also the first engine implemented in this project as a baseline (`faq_engine.py`), and running it shows both strengths and weaknesses. For "I want to register for placement", it finds "How can I register for placements?" with a score of 100.00%, because the important words match after preprocessing. For "Where can I see examination information?", it returns "How can I apply for an examination?" with only 48.14%, because it cannot recognise that "examination information" is closer in meaning to examination notices. For "What is the weather today?", every score is 0.00%.

2.5 Limitations of Existing Approach

- **Dependence on exact vocabulary:** synonyms and paraphrases are not recognised.
- **Manual browsing:** static pages require reading and comparing entries.

- **No confidence information:** the user cannot judge how reliable a result is.
- **No safe rejection:** the closest result is returned even when nothing relevant exists.
- **No query records:** gaps in the available information remain unnoticed.

2.6 Proposed System

The proposed Smart College FAQ System is a retrieval-based question answering application. The student types a question in a Streamlit interface; the question is sent to a FastAPI backend, which passes it to the hybrid engine. The engine computes a TF-IDF cosine similarity and a semantic cosine similarity (all-MiniLM-L6-v2) against each of the 34 FAQ questions, combines them, adds a small boost for shared intent words, and selects the highest-scoring FAQ. If the score reaches 0.50, the answer, category, matched question and confidence are returned; otherwise a fallback message with category "Unknown" is returned. Every result is stored in SQLite and displayed to the student.

2.7 Proposed Solution

- **Semantic matching** handles paraphrased questions.
- **Lexical matching** retains exact matches on important words as strong evidence.
- **A weighted hybrid score** (0.30 lexical, 0.70 semantic) gives priority to meaning.
- **An intent-word boost** of 0.05 rewards shared words such as "hostel", "placement" and "register".
- **A threshold** of 0.50 rejects unrelated questions with a fallback message.
- **Confidence and matched FAQ display** make each answer transparent.
- **History logging** and **a REST API** support review and independent testing.

2.8 Comparison Between Existing and Proposed System

Criterion	Existing Approach (Static FAQ / Keyword Search / Manual)	Proposed Smart College FAQ System
Input method	Browse a list, type keywords or ask staff	Free-text question in natural language
Matching technique	Manual reading or keyword overlap	Hybrid of TF-IDF and all-MiniLM-L6-v2 similarity
Paraphrased questions	Poorly handled	Supported through sentence embeddings
Confidence indication	Not provided	Shown with every answer
Unrelated questions	Closest result or no result	Rejected below 0.50 with a fallback message
Availability	Office hours or static page	Whenever the application is running
Query records	Not recorded	Stored in SQLite (question_history)

Criterion	Existing Approach (Static FAQ / Keyword Search / Manual)	Proposed Smart College FAQ System
Answer source	Varies by person or notice	Fixed answers from the FAQ dataset

Table 2.1: Comparison Between Existing Approach and Proposed System

The proposed system does not replace official information sources; it provides a faster and more flexible way of reaching the answers stored in the dataset and is explicit about the cases it cannot answer.

CHAPTER 3 REQUIREMENT GATHERING AND ANALYSIS

3.1 Requirement Gathering

Requirements were gathered by identifying the types of questions students commonly ask and the way an automated system should respond. Topics were grouped into eleven categories (Admission, Examination, Fees, Library, Hostel, Attendance, Placement, Scholarship, Academic, Student Activities and General), and representative questions with general answers were written for each, forming the dataset described in Chapter 6. Behavioural requirements were derived from the problems in Chapter 2: the system must accept free text, tolerate differences in wording, return one clear answer with supporting information and avoid answering questions outside its knowledge.

3.2 Functional Requirements

ID	Requirement	Implemented In
FR1	Accept a question typed in natural language	app.py (st.text_input)
FR2	Provide popular question shortcuts for Library, Hostel and Placement	app.py (st.button)
FR3	Warn the user if the question box is empty	app.py (st.warning)
FR4	Send the question to the backend as JSON over HTTP	app.py (requests.post)
FR5	Preprocess the question (lowercase, remove non-letters, tokenize, remove stopwords, lemmatize)	preprocessing.py
FR6	Compute TF-IDF and semantic (all-MiniLM-L6-v2) cosine similarity with every FAQ	hybrid_engine.py
FR7	Combine scores as 0.30 TF-IDF + 0.70 semantic, add 0.05 intent boost, clip to 0–1	hybrid_engine.py
FR8	Return answer, matched question, category and confidence when score ≥ 0.50; otherwise a fallback with category Unknown	hybrid_engine.py, main.py
FR9	Store each processed question with answer, category, confidence and status	database.py
FR10	Provide endpoints for FAQs, categories, history and health	backend/main.py
FR11	Display answer, category, confidence and matched FAQ, or a warning	app.py
FR12	Show an error message when the backend is unavailable or slow	app.py

Table 3.1: Functional Requirements

3.3 Non-Functional Requirements

Attribute	Requirement
Usability	One input box, one button and clearly labelled results; usable without instructions
Responsiveness	Model and FAQ embeddings loaded once at startup; frontend waits up to 60 seconds before reporting a timeout (response time not formally measured)
Reliability	Empty input, unavailable backend and request errors produce clear messages instead of crashes
Transparency	Every answer shown with its category, confidence and matched FAQ
Safety of answers	A fallback message is preferred over a low-confidence answer (threshold 0.50)
Maintainability	Preprocessing, retrieval, API, database and interface in separate modules
Portability	Runs on any platform supporting Python and the required libraries

Table 3.2: Non-Functional Requirements

3.4 Hardware Requirements

The system does not train any model and the code does not configure a GPU; the pretrained model is used only for encoding sentences on the CPU. The development environment in the screenshots is a Windows computer running VS Code with Python 3.13 (64-bit). Exact minimum values were not measured, so Table 3.3 describes the type of hardware required.

Component	Requirement
Processor	64-bit x86 CPU capable of running Python 3.13 and Sentence Transformers
GPU	Not required by the current implementation
Memory	Enough to load the Sentence Transformer model and run the two servers (minimum not measured)
Storage	Space for the virtual environment, libraries, cached model, dataset and database file
Network	Internet for the first download of the model and NLTK resources; afterwards local address 127.0.0.1

Table 3.3: Hardware Requirements

3.5 Software Requirements

Library versions are not listed because the project's requirements.txt was not part of the analysed material.

Software	Purpose
Operating system	Windows (used in development) or any OS supporting Python
Python 3.13	Programming language for all components
Frameworks	Streamlit (UI); FastAPI, Pydantic and Uvicorn (API)
NLP libraries	NLTK, scikit-learn, Sentence Transformers (all-MiniLM-L6-v2)
Data	pandas (dataset), SQLite via sqlite3 (history)
Other	Requests (HTTP client), VS Code (editor), web browser

Table 3.4: Software Requirements

3.6 User Requirements

The primary user is a student; a secondary user is a developer or tester who accesses the API directly through Swagger or `test_api.py`.

- The student must be able to type a question and submit it with one button, or select a popular question.
- The student must see the answer, its category, the confidence and the matched FAQ question.
- The student must be told clearly when no answer is available and what to do next.
- The developer must be able to call `/ask`, list FAQs and categories, view history and check service health.

3.7 System Constraints

- Only answers present in `data/faq_data.csv` can be returned.
- The stopword list, lemmatizer and semantic model are used for English text.
- The frontend is configured with `API_URL = "http://127.0.0.1:8000"`, so both parts run on the same machine.
- The backend must be started from the project root before the frontend is used, because it uses relative paths for the dataset and database.
- The model and NLTK resources must be downloaded once before offline use.

3.8 Feasibility Analysis

The system is technically feasible because it uses mature open-source libraries and a pretrained model (no training or GPU), and the screenshots and stored history confirm it runs on the development machine. It is economically feasible because all tools are free, and operationally feasible because the interface needs no technical knowledge, although practical use requires maintaining the dataset with verified information.

CHAPTER 4 SYSTEM DESIGN AND ARCHITECTURE

4.1 System Overview

The Smart College FAQ System is a three-tier application. The presentation tier is the Streamlit application used in a web browser; the application tier is the FastAPI backend containing the hybrid retrieval engine; and the data tier consists of the FAQ dataset (CSV file) and the question history (SQLite database). A student uses the Streamlit interface, while a developer or tester can use the REST endpoints directly. Figure 4.1 shows the use cases of these two actors.

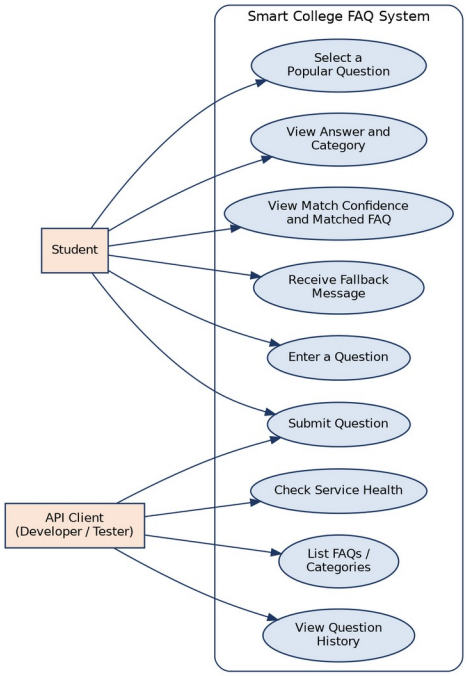


Figure 4.1: Use Case Diagram of the Smart College FAQ System

4.2 Overall Architecture

Figure 4.2 presents the overall architecture. A question entered in Streamlit is sent to FastAPI as an HTTP POST request with a JSON body. The backend validates it with a Pydantic model, calls `find_answer()` of the `HybridFAQEngine`, saves the result in SQLite and returns a JSON response. The engine is created once when the backend starts, because loading the Sentence Transformer model and encoding the FAQ questions are the most expensive operations; each request then needs only preprocessing, one TF-IDF transformation, one sentence encoding and two similarity computations.

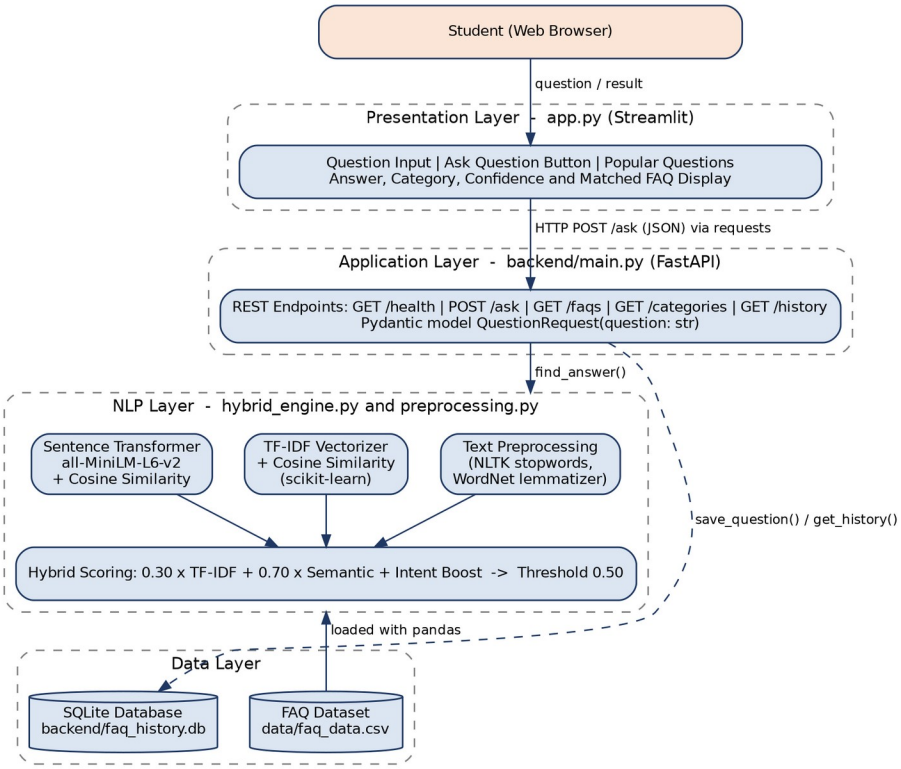


Figure 4.2: Overall System Architecture

4.3 System Components

Component	File(s)	Responsibility
User Interface	app.py	Collects the question, calls the API, displays results and errors
REST API	backend/main.py	FastAPI application with five endpoints; loads the engine at startup
Hybrid Engine	hybrid_engine.py	TF-IDF and semantic similarity, intent boost, threshold
Preprocessing	preprocessing.py	Lowercasing, character removal, tokenization, stopwords, lemmatization
TF-IDF Baseline	faq_engine.py	Lexical-only engine used for testing and comparison
Database Module	backend/database.py	Creates, inserts into and reads the question_history table
FAQ Dataset	data/faq_data.csv	34 records with id, category, question and answer
Test Scripts	test_*.py	Validation of preprocessing, retrieval and API behaviour

Table 4.1: System Components and Their Responsibilities

4.4 Data Flow

Figure 4.3 shows the context-level data flow diagram, in which the system exchanges data with two external entities. Figure 4.4 decomposes it into six processes: accepting the question (1.0), receiving and validating it (2.0), preprocessing (3.0), computing hybrid similarity against data store D1 (4.0), selecting the best FAQ and applying the threshold (5.0), and logging the result in D2 (6.0).

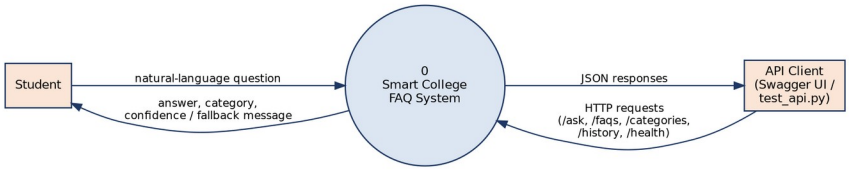


Figure 4.3: Context-Level Data Flow Diagram (Level 0)

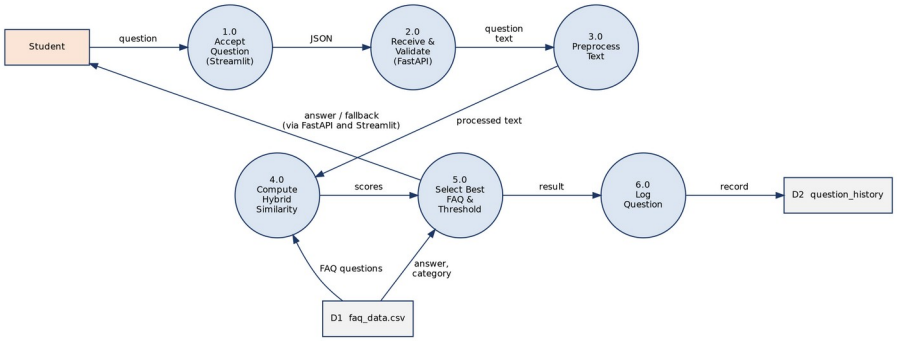


Figure 4.4: Level-1 Data Flow Diagram

4.5 NLP Processing Pipeline

Figure 4.5 shows the pipeline applied to every question. The same preprocessing is applied to the FAQ questions when the engine is created, so both sides of the TF-IDF comparison are produced identically. Preprocessing is used only for the TF-IDF branch; the semantic model receives the original text, because sentence embedding models benefit from the function words and word order that preprocessing removes.

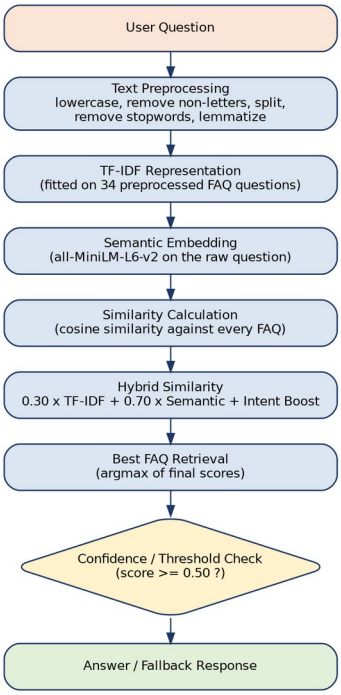


Figure 4.5: NLP Processing Pipeline

4.6 Query Processing

Figure 4.6 shows the complete handling of one question. Validation occurs at two levels: the frontend warns about an empty input without contacting the backend, and the backend returns "Please enter a question." for input that is empty after stripping, without calling the engine or saving a record. Every engine result, answer or fallback, is saved before the response is returned.

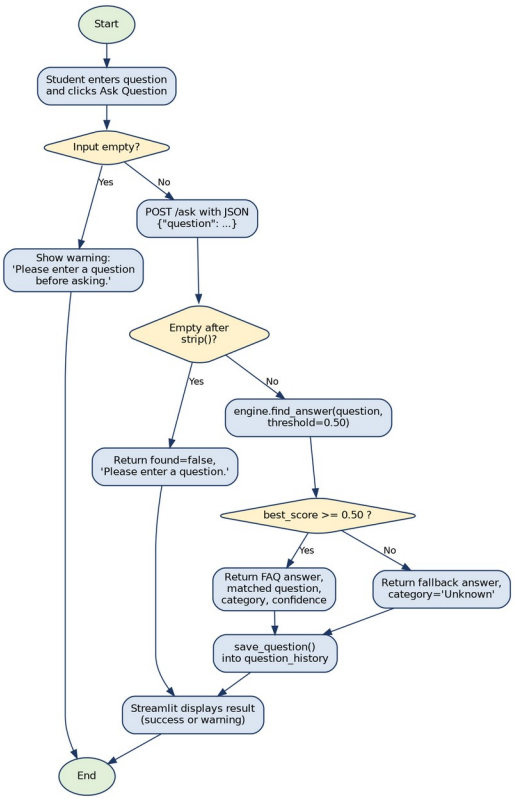


Figure 4.6: Query Processing Flowchart

4.7 FAQ Retrieval Process

For each question, the engine produces a final score for each of the 34 FAQ records. The index of the largest score identifies the selected record, and the largest score becomes the confidence value. There is no prior filtering by category; the returned category is simply that of the selected record, or "Unknown" if the score is below the threshold. Because every record is scored, a low confidence means that no record is similar to the question, which is exactly the condition used to detect unrelated questions.

4.8 Hybrid Similarity Architecture

Figure 4.7 shows the hybrid computation in `hybrid_engine.py`. The lexical branch preprocesses the question, applies the fitted TF-IDF vectorizer and computes cosine similarity against the FAQ TF-IDF matrix. The semantic branch encodes the raw question and computes cosine similarity against the precomputed 34 × 384 FAQ embedding matrix. The scores are combined with weights 0.30 and 0.70, the intent boost is added, the result is clipped to 0–1, and the best score is compared with the threshold. The boost is deliberately small so that it can reorder close candidates without making an unrelated FAQ a confident match.

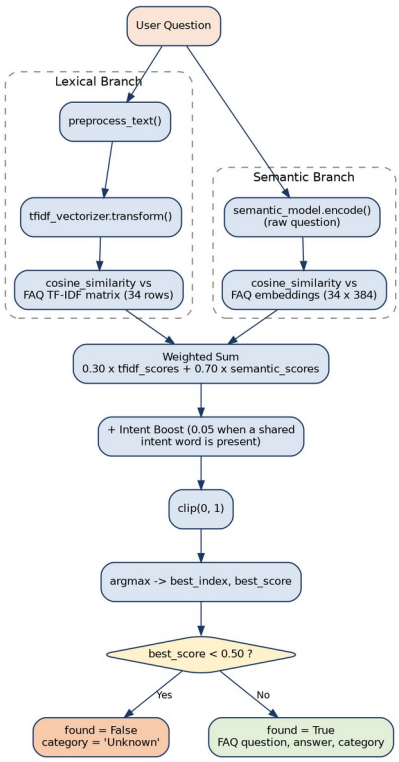


Figure 4.7: Hybrid Similarity Architecture

4.9 Backend Architecture

When `backend/main.py` is loaded, it prints "Initializing FAQ system...", creates the engine with `data/faq_data.csv`, calls `initialize_database()` and prints "FAQ backend ready!" (Figure 4.8). Endpoint functions handle HTTP concerns, the engine handles NLP logic and the database module handles persistence, which keeps each part independently testable.

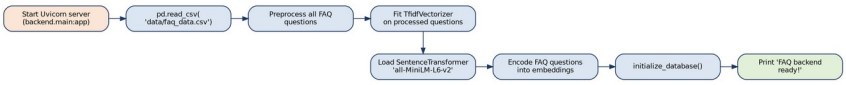


Figure 4.8: Backend Initialization Sequence

4.10 Frontend Architecture

The frontend is the single Streamlit script `app.py`, which Streamlit re-executes on each interaction. It is organised into sections for page configuration, backend URL, session state, CSS, header, question input, ask button, popular questions, result processing and footer. It contains no NLP logic and communicates with the backend only through `/ask`, as shown in Figure 4.9.

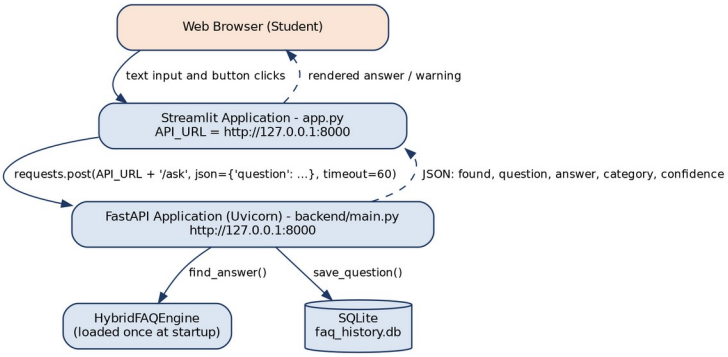
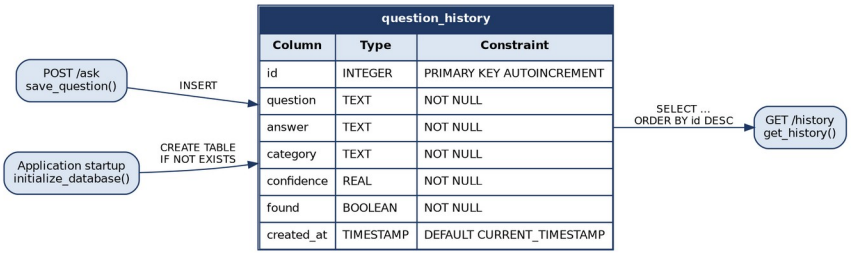


Figure 4.9: Frontend–Backend Communication Architecture

4.11 Database Architecture

A single SQLite file, `backend/faq_history.db`, contains one table, `question_history`, with one row per question processed through `/ask` (Figure 4.10). The FAQ data remains in the CSV file. `id` and `created_at` are filled automatically, and each database function opens and closes its own connection.



question_history

Column	Type	Constraint
id	INTEGER	PRIMARY KEY AUTOINCREMENT
question	TEXT	NOT NULL
answer	TEXT	NOT NULL
category	TEXT	NOT NULL
confidence	REAL	NOT NULL
found	BOOLEAN	NOT NULL
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

Figure 4.10: Database Schema and Access Operations

4.12 API Architecture

The API follows the REST style with five endpoints: GET `/health`, POST `/ask`, GET `/faqs`, GET `/categories` and GET `/history` (Table 9.1). FastAPI also generates an OpenAPI description and a Swagger interface at `/docs` (Chapter 9). All endpoints return JSON, and only `/ask` modifies data, by inserting a history record.

CHAPTER 5 TECHNOLOGY STACK

This chapter describes only the technologies that appear in the project source code or screenshots; Table 3.4 summarises them.

5.1 Python

Python is the only language used, covering NLP, the API, the interface and storage. The screenshots show Python 3.13 (64-bit) in a virtual environment.

5.2 Streamlit

Streamlit builds interactive web applications from Python scripts, re-running the script on each widget interaction and keeping persistent values in `st.session_state`. In `app.py` it provides the page configuration, the text input, buttons, columns and result elements (`st.success`, `st.warning`, `st.info`), with custom CSS injected through `st.markdown(..., unsafe_allow_html=True)`.

5.3 FastAPI

FastAPI defines API endpoints as decorated Python functions, validates request bodies with Pydantic models, converts returned dictionaries to JSON and generates OpenAPI documentation with a Swagger interface. In `backend/main.py` the application is created with the title "Smart College FAQ API" and version "1.0.0", and the `/ask` body is validated by the `QuestionRequest` model. Response headers captured in Swagger show `server: uvicorn`, indicating that the application is served by the Uvicorn ASGI server.

5.4 Natural Language Processing

This project addresses FAQ retrieval, an NLP task of finding the stored question most similar to a user's question. It combines classical techniques (normalisation, stopword removal, lemmatization, TF-IDF, cosine similarity) with neural sentence embeddings.

5.5 NLTK

The Natural Language Toolkit provides corpora and text processing tools. `preprocessing.py` uses the English stopword list from `nltk.corpus.stopwords` and the `WordNetLemmatizer`, which reduces words to base forms using the WordNet lexical database (for example, "placements" to "placement"). The resources `stopwords`, `wordnet` and `omw-1.4` are downloaded with `nltk.download` when the module is imported.

5.6 TF-IDF

TF-IDF represents a text as a vector of term weights that increase with the frequency of a term in the text and decrease with the number of texts containing it, so distinctive terms such as "scholarship" receive higher weights than common ones. The project uses scikit-learn's `TfidfVectorizer` with default settings, fitted on the preprocessed text of the 34 FAQ questions. The formulation is given in Chapter 6.

5.7 Cosine Similarity

Cosine similarity measures the cosine of the angle between two vectors, giving 1 for identical directions and 0 for orthogonal vectors. It depends on direction rather than length, which suits texts of different lengths. The project uses `sklearn.metrics.pairwise.cosine_similarity` for both TF-IDF vectors and embeddings, comparing each question with all FAQ vectors in one call.

5.8 Sentence Transformers

Sentence Transformers is a library for computing sentence embeddings with transformer models trained so that sentences with similar meanings lie close together in vector space (the Sentence-BERT approach). `hybrid_engine.py` loads the model with `SentenceTransformer("all-MiniLM-L6-v2")` and encodes text with `encode(..., convert_to_numpy=True)`. The library is also used in `test_semantic.py`.

5.9 all-MiniLM-L6-v2

all-MiniLM-L6-v2 is a pretrained sentence embedding model that maps a sentence to a 384-dimensional vector. It is based on MiniLM, a compact transformer produced by knowledge distillation, with six layers, which makes CPU use practical. The project uses it without fine-tuning: the FAQ questions are encoded once into a 34 × 384 matrix, and each incoming question is encoded at query time.

5.10 Pandas

pandas loads the dataset with `pd.read_csv()`; the DataFrame gains a `processed_question` column and feeds the `/faqs` and `/categories` responses.

5.11 Scikit-learn

scikit-learn is used only for `TfidfVectorizer` and `cosine_similarity`; no model is trained.

5.12 SQLite

SQLite stores a relational database in a single file without a server process and is available through Python's `sqlite3` module. `backend/database.py` uses it to create the `question_history` table, insert one record per question and read history, with `row_factory = sqlite3.Row` so rows can be converted to dictionaries.

5.13 Requests

Requests is an HTTP client library. `app.py` uses `requests.post(f"{API_URL}/ask", json={...}, timeout=60)`, checks the status with `raise_for_status()` and handles `ConnectionError`, `Timeout` and `RequestException`. `test_api.py` uses it to send the validation questions.

5.14 Development Environment

The project folder `Smart-College-FAQ` was developed on Windows with a virtual environment activated in PowerShell. The backend runs at `127.0.0.1:8000` with Swagger at `/docs`, and `requirements.txt` records the dependencies.

5.15 VS Code

VS Code was used for editing, browsing the project and running scripts in its terminal (Figure F.1).

CHAPTER 6 DATASET AND NLP PROCESSING

6.1 Dataset Overview

The system's knowledge is stored in `data/faq_data.csv`, which contains 34 FAQ records. Each record has a question a student might ask, a general answer and a category. The answers direct students to the relevant procedure or office (for example, "Students should use the fee payment procedure specified by the college.") and are sample answers for demonstration, not verified official policies. Because the system is retrieval-based, the dataset defines its entire scope. Extending its knowledge only requires editing the dataset, since the TF-IDF matrix and FAQ embeddings are rebuilt each time the backend starts.

6.2 FAQ Dataset Structure

The dataset is a CSV file with a header row and one row per record; answers are enclosed in double quotes so that commas inside them do not break the format. The complete dataset is listed in Table B.1 (Appendix B).

6.3 Dataset Fields

Field	Type	Description and Use
id	Integer	Unique identifier (1 to 34); returned by /faqs
category	Text	One of 11 categories; returned with the answer
question	Text	Preprocessed for TF-IDF and encoded in raw form for semantic similarity
answer	Text	Returned when the record is the best match
processed_question	Text (in memory)	Created by preprocess_text() at engine start-up; not stored in the CSV

Table 6.1: Dataset Fields

6.4 FAQ Categories

The 34 records are distributed across 11 categories (Table 6.2 and Figure 6.1). Library and Hostel have the most records because several phrasings of the same need were included; General has one.

Category	Records	Example Question
Admission	3	What is the admission process?
Examination	4	Where can I check the examination schedule?
Fees	2	How can I pay my college fees?
Library	6	How can I borrow a book from the library?
Hostel	6	How can I apply for hostel accommodation?
Attendance	2	What is the attendance requirement?

Category	Records	Example Question
Placement	3	How can I register for placements?
Scholarship	2	How can I apply for a scholarship?
Academic	3	How can I contact my department?
Student Activities	2	How can I join a college club?
General	1	What are the college working hours?

Table 6.2: FAQ Categories and Record Counts

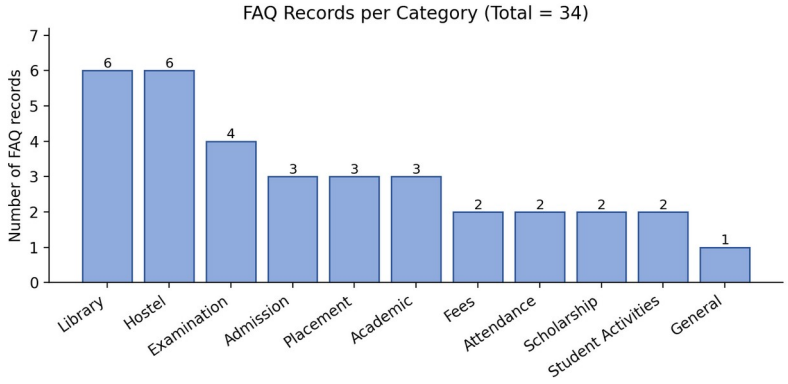


Figure 6.1: Distribution of FAQ Records Across Categories

6.5 Data Preprocessing

The function `preprocess_text()` applies five steps in sequence, illustrated in Figure 6.2 on a question from `test_preprocessing.py`. It is applied to the FAQ questions at start-up and to every incoming question; its output is used only for TF-IDF and the intent boost.

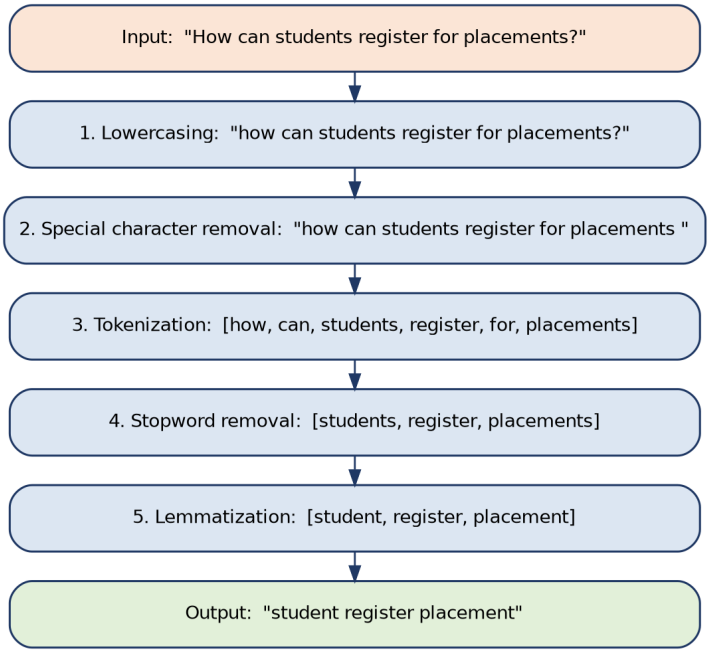


Figure 6.2: Preprocessing Steps Applied to a Sample Question

6.6 Lowercasing

All characters are converted to lowercase with `str.lower()`, so "Hostel" and "hostel" become the same term.

6.7 Special Character Removal

Every character that is not an English letter or whitespace is replaced with a space using `re.sub(r"[^a-zA-Z\s]", " ", text)`, removing punctuation and digits without merging adjacent words.

6.8 Tokenization

The text is split into tokens with `str.split()`, which splits on whitespace. Since special characters are already replaced, this is sufficient; for example, "how can students register for placements " becomes [how, can, students, register, for, placements].

6.9 Stopword Removal

Tokens found in NLTK's English stopword set (such as "how", "can", "i", "the", "for") are removed, leaving content words. For the unrelated question "What is the weather today?", only "weather" and "today" remain, and neither occurs in the FAQ vocabulary, so its TF-IDF similarity with every FAQ is zero.

6.10 Lemmatization

Each remaining token is reduced to its base form with NLTK's `WordNetLemmatizer`. Without a part-of-speech argument, words are treated as nouns, so plurals are singularised ("students" to

"student", "hours" to "hour") while verb forms such as "working" and "required" are unchanged. Table 6.3 lists the outputs of `test_preprocessing.py`.

Original Question	Processed Output
How can I apply for the examinations?	apply examination
Where can I find the college library?	find college library
What are the working hours of the college?	working hour college
How can students register for placements?	student register placement

Table 6.3: Preprocessing Outputs from test_preprocessing.py

6.11 TF-IDF

Fitted on the 34 preprocessed FAQ questions, the vectorizer builds a vocabulary of 44 terms. With scikit-learn's default settings, term frequency is the raw count, inverse document frequency is smoothed, and each vector is L2-normalised, where n = 34 is the number of FAQ questions and df(t) is the number containing term t:

$$tf(t, d) = number\ of\ occurrences\ of\ term\ t\ in\ document\ d \tag{6.1}$$

$$idf(t) = \ln[\,(1 + n)/(1 + df(t))\,] + 1 \tag{6.2}$$

$$w(t, d) = tf(t, d) \times idf(t), \quad v(d) = w(d)/\|w(d)\|_2 \tag{6.3}$$

Rare terms receive higher weights: "register", which occurs in one FAQ question, has idf = 3.8622, while "hostel", which occurs in six, has idf = 2.6094.

6.12 Cosine Similarity

The similarity between the question vector q and an FAQ vector f is:

$$\cos(q, f) = (q \cdot f)/(\,\|q\| \times \|f\|\,) \tag{6.4}$$

Because TF-IDF vectors are L2-normalised and non-negative, this reduces to the dot product and lies between 0 and 1. For example, "How can I pay my college fees?" and the identically worded FAQ both become "pay college fee", giving a TF-IDF similarity of 1.0.

6.13 Semantic Embeddings

A semantic embedding is a dense vector representing the meaning of a sentence; sentences with similar meanings are close even when they share few words. Each FAQ question is encoded into a 384-dimensional vector, forming a matrix E of size 34 × 384. For an incoming question with embedding e(q), the semantic similarity with FAQ i is:

$$S_i = \cos(\,e(q), E_i\,), \quad i = 1, \ldots, 34 \tag{6.5}$$

6.14 Sentence Transformer

The embeddings are produced by all-MiniLM-L6-v2, which passes the tokens of a sentence through a transformer network and pools the contextual token vectors into one sentence vector. The model is pretrained and used without fine-tuning, and it receives the original question, because words such as "where" and "how" help indicate the type of information requested.

6.15 Hybrid Similarity

The base hybrid score combines the TF-IDF similarity T_i and the semantic similarity S_i:

$$H_i = 0.30 \times T_i + 0.70 \times S_i \tag{6.6}$$

Let U and F_i be the word sets of the preprocessed question and FAQ i, and K the set of 18 intent words in Table 6.4. A boost is added and the result clipped:

$$B_i = 0.05 \text{ if } (U \cap F_i \cap K) \neq \varnothing, \text{ otherwise } 0 \tag{6.7}$$

$$C_i = \min(1, \max(0, H_i + B_i)) \tag{6.8}$$

Intent Words (set K in hybrid_engine.py)
apply, application, admission, register, registration, issue, borrow, payment, pay, schedule, requirement, information, notice, accommodation, hostel, scholarship, placement, attendance

Table 6.4: Intent Words Used for the Score Boost

Clipping explains why questions worded exactly like a dataset question receive 100.00%: both similarities are close to 1, and any boost pushes the sum above 1. Table 6.5 lists all retrieval parameters.

Parameter	Value	Defined In
TF-IDF / semantic weights	0.30 / 0.70	hybrid_engine.py
Intent boost	0.05	hybrid_engine.py
Hybrid acceptance threshold	0.50	hybrid_engine.py; passed in main.py
TF-IDF baseline threshold	0.20	faq_engine.py
Embedding dimension	384	all-MiniLM-L6-v2

Table 6.5: Retrieval Parameters Used in the Project

6.16 Threshold-Based Retrieval

The best FAQ and the decision are given by:

$$i^* = \text{argmax}_i \, C_i; \quad found = true \text{ if } C_i^* \geq 0.50, \text{ otherwise false} \tag{6.9}$$

For related questions both similarities are usually high for one record, and in the validation set all eleven related questions scored at least 82.33%. For unrelated questions TF-IDF is usually zero everywhere, so the confidence is at most 0.70 times the highest semantic similarity; the three

unrelated validation questions scored 10.33% to 15.85%. The threshold lies in the wide gap between these groups. The value 0.50 was chosen by the team and not tuned on a separate dataset.

6.17 Unknown Question Handling

When the best score is below 0.50, the engine returns `found` = False, `question` = None, category "Unknown", the rejected best score as confidence and the answer "Sorry, I could not find a relevant answer. Please contact the concerned college department." For example, "What is the weather today?" received 0.1129. The frontend then shows a warning instead of an answer.

CHAPTER 7 SYSTEM IMPLEMENTATION

7.1 Implementation Overview

The design of Chapter 4 was implemented as a set of Python modules, each with a test script. Figure 7.1 shows the dependencies: the dataset feeds the preprocessing and retrieval modules, the TF-IDF and semantic techniques are combined in the hybrid engine, the backend uses the hybrid engine, and both the frontend and the API test use the backend. Code excerpts in this chapter are taken from the project files; shortened excerpts are marked, and principal files are reproduced in Appendix C.

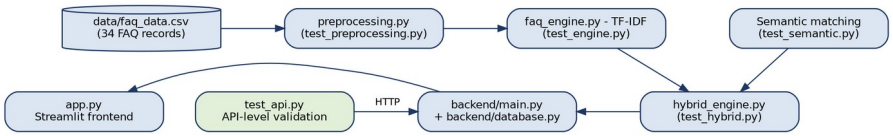


Figure 7.1: Module Dependency and Testing Workflow

7.2 Project Structure

File / Folder	Description
app.py	Streamlit frontend
faq_engine.py	TF-IDF FAQEngine (lexical baseline)
hybrid_engine.py	HybridFAQEngine (TF-IDF + semantic)
preprocessing.py	preprocess_text() and NLTK resource loading
backend/__init__.py	Makes backend a package so main.py can import backend.database
backend/main.py, backend/database.py	FastAPI application; SQLite functions
backend/faq_history.db	SQLite database created at runtime
data/faq_data.csv	FAQ dataset (34 records)
test_preprocessing.py, test_engine.py, test_semantic.py, test_hybrid.py, test_api.py	Test scripts (Chapter 10)
requirements.txt, venv/, models/, screenshots/	Dependency list, virtual environment and supporting folders

Table 7.1: Project File Structure

7.3 Preprocessing Implementation

The module loads the English stopwords into a set for fast lookup and creates one reusable `WordNetLemmatizer`. Lemmatization is applied only to tokens that survive stopword removal, and the function returns a string because `TfidfVectorizer` expects strings.

```
stop_words = set(stopwords.words("english"))
lemmatizer = WordNetLemmatizer()

def preprocess_text(text):
    text = text.lower()
    text = re.sub(r"[^a-zA-Z\s]", " ", text)
    words = text.split()
    processed_words = []
    for word in words:
        if word not in stop_words:
            lemma = lemmatizer.lemmatize(word)
            processed_words.append(lemma)
    return " ".join(processed_words)
```

Code Snippet 7.1: Text Preprocessing Function (preprocessing.py)

7.4 FAQ Engine Implementation

`FAQEngine` loads the dataset, adds `processed_question` and fits a `TfidfVectorizer`. `find_top_matches()` sorts the cosine scores with `argsort()[::-1][:top_k]` to obtain the k best records, and `find_answer()` applies a threshold of 0.20 and also returns the top three matches. This engine is not used by the backend; it serves as a baseline for testing and for the comparison in Chapter 11.

7.5 Hybrid Engine Implementation

The `HybridFAQEngine` constructor prepares both FAQ representations: the TF-IDF matrix of the preprocessed questions and the embedding matrix of the raw questions.

```
self.faq_data = pd.read_csv(dataset_path)
self.faq_data["processed_question"] = self.faq_data["question"].apply(preprocess_text)
self.tfidf_vectorizer = TfidfVectorizer()
self.tfidf_vectors =
self.tfidf_vectorizer.fit_transform(self.faq_data["processed_question"])
self.semantic_model = SentenceTransformer("all-MiniLM-L6-v2")
self.semantic_embeddings = self.semantic_model.encode(
    self.faq_data["question"].tolist(), convert_to_numpy=True)
```

Code Snippet 7.2: Hybrid Engine Initialisation (hybrid_engine.py, shortened)

`find_answer()` combines the two similarity vectors, adds the boost from `calculate_intent_boost()` (which intersects the preprocessed word sets of both questions with the 18 intent words and returns 0.05 or 0.0), clips the scores and applies the threshold.

```
hybrid_scores = (0.30 * tfidf_scores + 0.70 * semantic_scores)
final_scores = []
for index in range(len(self.faq_data)):
    boost = self.calculate_intent_boost(
        user_question, self.faq_data.iloc[index]["question"])
    final_scores.append(hybrid_scores[index] + boost)
final_scores = pd.Series(final_scores).to_numpy().clip(0, 1)
best_index = final_scores.argmax()
best_score = final_scores[best_index]
if best_score < threshold:
    return {"found": False, "question": None,
            "answer": ("Sorry, I could not find a relevant answer. "
                       "Please contact the concerned college department."),
            "category": "Unknown", "confidence": float(best_score)}
```

Code Snippet 7.3: Hybrid Scoring and Threshold Check (hybrid_engine.py, shortened)

Because `calculate_intent_boost()` preprocesses both questions on every call, each request repeats preprocessing 34 times; this is noted as an optimisation in Chapter 12.

7.6 Semantic Model Integration

The model name "all-MiniLM-L6-v2" is passed to `SentenceTransformer`, which downloads and caches the model on first use. `encode()` is called once at start-up for the 34 FAQ questions and once per request for the incoming question, with `convert_to_numpy=True` so the result can be passed directly to `cosine_similarity`. `test_semantic.py` exercises the model in isolation before it is combined with TF-IDF.

7.7 FastAPI Backend Implementation

The backend creates the application, the engine and the database table at import time, so these happen once when the server starts. The `QuestionRequest` model requires a string field `question`; non-matching requests are rejected automatically.

```
app = FastAPI(title="Smart College FAQ API", description=(...), version="1.0.0")


engine = HybridFAQEngine("data/faq_data.csv")
initialize_database()


class QuestionRequest(BaseModel):
    question: str
```

Code Snippet 7.4: Application Creation and Start-up (backend/main.py, shortened)

7.8 API Endpoint Implementation

The `/ask` endpoint strips the question, handles empty input, calls the engine with threshold 0.50, saves the result and returns five fields. The read-only endpoints return the DataFrame records (`/faqs`), the sorted unique categories (`/categories`), the stored history (`/history`) and a fixed status (`/health`).

```
@app.post("/ask")
def ask_question(request: QuestionRequest):
    question = request.question.strip()
    if not question:
        return {"found": False, "answer": "Please enter a question.",
                "category": "Unknown", "confidence": 0.0}
    result = engine.find_answer(question, threshold=0.50)
    save_question(question=question, answer=result["answer"],
                  category=result["category"],
                  confidence=result["confidence"], found=result["found"])
    return {"found": result["found"], "question": result["question"],
            "answer": result["answer"], "category": result["category"],
            "confidence": result["confidence"]}
```

Code Snippet 7.5: The /ask Endpoint (backend/main.py)

7.9 Database Implementation

`database.py` defines `DATABASE_PATH = Path("backend/faq_history.db")`. `initialize_database()` uses `CREATE TABLE IF NOT EXISTS`, so existing records survive restarts; `save_question()` inserts a row with a parameterised query; and `get_history()` returns all rows ordered by id descending. The `?` placeholders pass the question text as a parameter, so quotation marks or SQL keywords in a question cannot alter the statement. SQLite stores the BOOLEAN `found` value as 1 or 0.

```
cursor.execute(
    """INSERT INTO question_history
       (question, answer, category, confidence, found)
       VALUES (?, ?, ?, ?, ?)""",
    (question, answer, category, confidence, found))
connection.commit()
connection.close()
```

Code Snippet 7.6: Saving a Question Record (backend/database.py, shortened)

7.10 Streamlit Frontend Implementation

`app.py` sets `API_URL = "http://127.0.0.1:8000"`, initialises `st.session_state.question`, and creates the input box (placeholder "Example: How can I apply for hostel?"), the Ask Question button and three popular question buttons. Each popular button stores a predefined question ("Where can I get books?", "How can I get a hostel room?", "How do I register for placements?") in the session state and calls `st.rerun()`. When a result arrives, a found result is shown with a success message, the answer in a styled box, the category, the confidence (multiplied by 100, two decimals) and the matched FAQ; otherwise a warning and guidance text are shown.

7.11 Backend-Frontend Communication

When Ask Question is clicked with non-empty input, the frontend sends `POST http://127.0.0.1:8000/ask` with body `{"question": "<text>"}` and a 60-second timeout, parses the JSON response and renders its fields. Because the parts communicate only through this interface, the backend can be tested independently, as `test_api.py` does.

7.12 Error Handling

Empty input is caught in both the frontend ("Please enter a question before asking.") and the backend, and invalid bodies are rejected by FastAPI. In the frontend, `ConnectionError` produces "The FAQ service is currently unavailable. Please make sure the FastAPI backend is running.", `Timeout` produces "The request took too long. Please try again.", and other `RequestException` errors produce a general retry message. In `test_api.py`, each request is wrapped in try/except, so a failed request counts as FAIL while the other tests continue (Figure F.1).

7.13 Unknown Query Handling

The engine returns the fallback result below the threshold (Code Snippet 7.3), the backend stores it with category "Unknown" and `found` = 0, and the frontend shows a warning instead of an answer. The TF-IDF baseline differs from the hybrid engine in using only lexical similarity, no boost or clipping, a threshold of 0.20 and returning the top three matches.

CHAPTER 8 USER INTERFACE DESIGN

8.1 UI Overview

The user interface is a single-page Streamlit application built around one task: asking a question and reading the answer. Custom CSS sets a centred layout with a maximum width of 900 pixels and hides Streamlit's default menu and footer. Table 8.1 lists the components from top to bottom. The screenshots in this chapter were captured from the running application and keep their original aspect ratio.

Component	Streamlit Element	Purpose
Header	st.markdown (HTML/CSS)	Icon, title and subtitle "Your college questions, answered simply."
Question input	st.text_input	"Enter your question" with an example placeholder
Ask button	st.button (full width)	Sends the question to the backend
Popular questions	Three st.button in st.columns(3)	Library, Hostel and Placement shortcuts
Result status	st.success / st.warning	Shows whether an answer was found
Answer	st.subheader + styled box	Answer text
Category, confidence	st.caption + st.info	FAQ category and match confidence
Matched FAQ	st.caption + styled box	The stored FAQ question that was matched
Footer	st.markdown	Project name and tagline

Table 8.1: User Interface Components

8.2 Home Screen

Figure 8.1 shows the home screen. Below the header, the section "What would you like to know?" lists the supported topics, followed by the input box, the Ask Question button, the Popular Questions row and the footer. There are no menus, sidebars or extra pages, so a student can see immediately where to type and what to press.

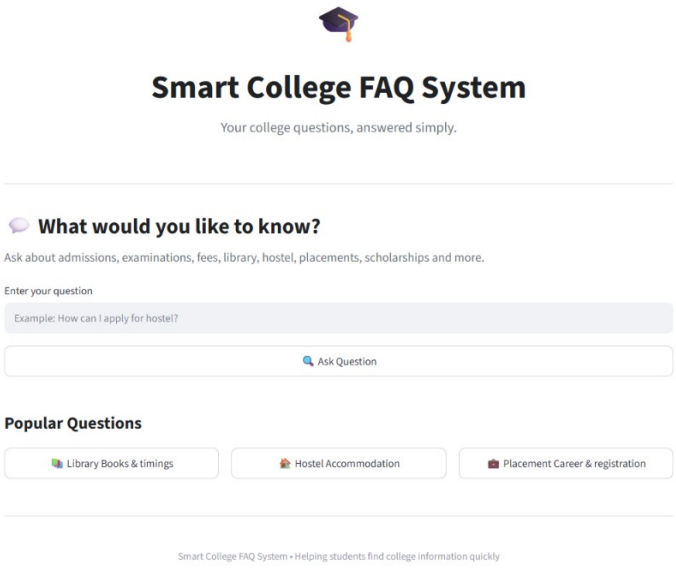


Figure 8.1: Smart College FAQ System Home Interface

8.3 Question Input

The input box shows the placeholder "Example: How can I apply for hostel?" (Figure 8.1). Any plain-English question can be typed without special syntax. On pressing Ask Question, leading and trailing spaces are removed; if nothing remains, the warning "Please enter a question before asking." is shown and no request is sent.

8.4 Popular Questions

The Popular Questions row has three buttons: Library (Books & timings), Hostel (Accommodation) and Placement (Career & registration). They are associated with "Where can I get books?", "How can I get a hostel room?" and "How do I register for placements?". Pressing a button stores the question in the session state and reruns the page; the student then presses Ask Question to submit it.

8.5 Answer Display

For a found result, the interface shows "✓ Relevant answer found", the heading "Answer" and the answer text in a light-blue box. Figure 8.2 shows the result for "Where do I get books?". Although the question does not contain the word "library", it was matched to "Where can I get books from the library?" with a confidence of 83.45%.

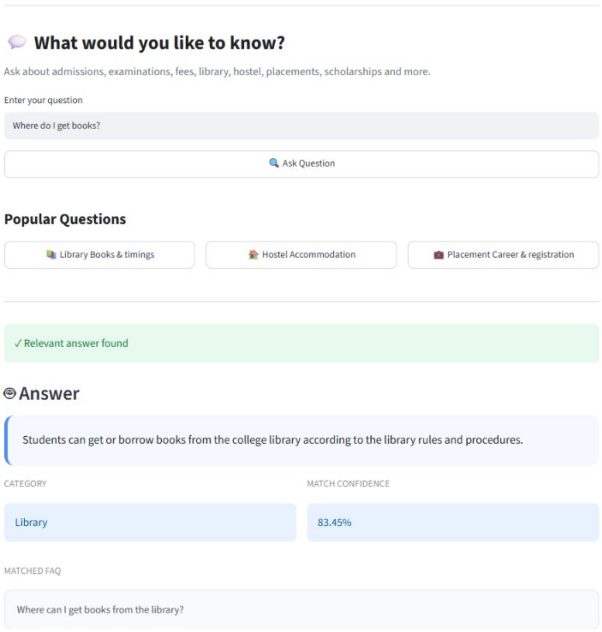


Figure 8.2: Library FAQ Response

Figure 8.3 shows "How can I get a hostel room?", which is identical to a dataset question and received 100.00%. Figure 8.4 shows "I want to register for campus placement", a statement with the extra word "campus", matched to "How can I register for placements?" with 90.75%.

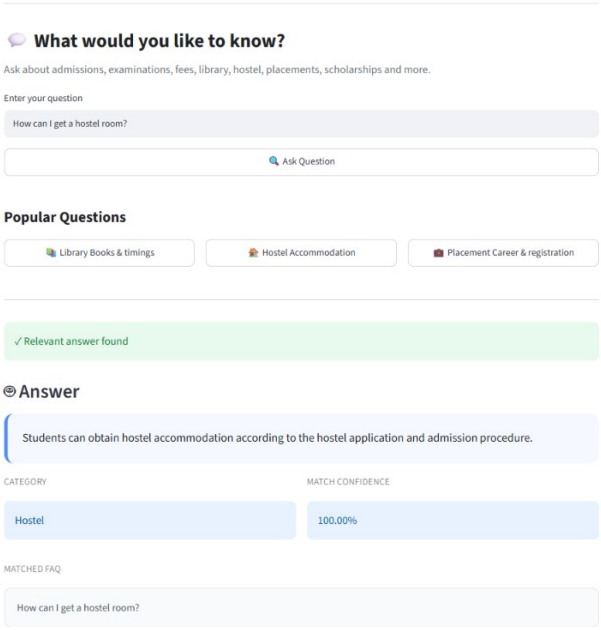


Figure 8.3: Hostel FAQ Response

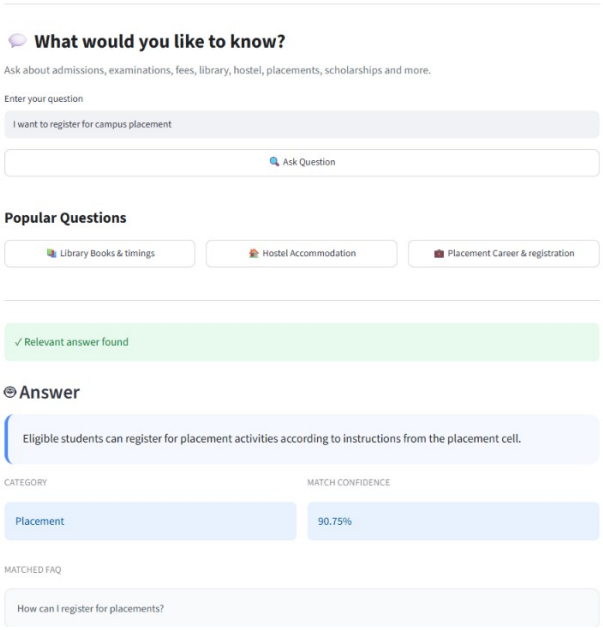


Figure 8.4: Placement FAQ Response

8.6 Category Display

The matched FAQ's category is shown under "CATEGORY" (Library, Hostel and Placement in Figures 8.2 to 8.4), confirming that the topic was understood.

8.7 Confidence Display

The confidence is shown under "MATCH CONFIDENCE". The backend returns a value between 0 and 1, which the frontend multiplies by 100 and formats with two decimals. 100.00% indicates an exact or near-exact match; lower values indicate a match on meaning rather than identical wording. Confidence is not displayed for unknown questions, although the backend still returns it.

8.8 Matched FAQ Display

The stored question that was selected is shown under "MATCHED FAQ". If it differs from what the student meant, the student can recognise this and rephrase, rather than relying on an answer to a different question.

8.9 Unknown Question Response

For a result with `found` = false, no answer, category, confidence or matched FAQ is shown. Instead, a yellow warning "We couldn't find a relevant answer." appears with the advice to rephrase the question or contact the concerned college department. Figure 8.5 shows this for "What is the weather today?", which received a confidence of 11.29% in the backend.

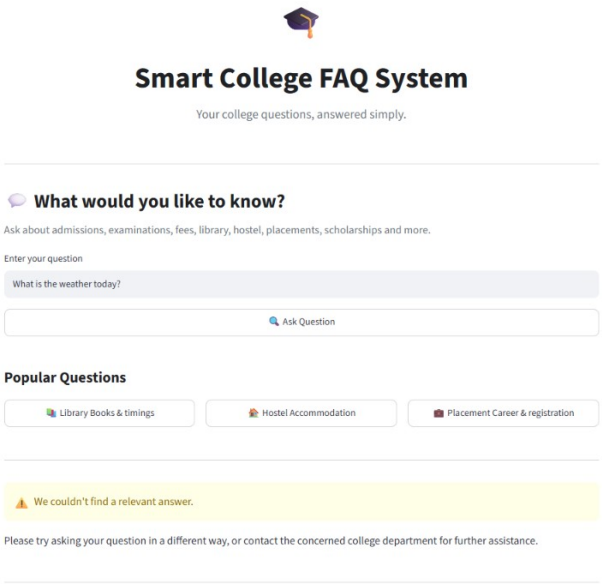


Figure 8.5: Unknown Query Handling in the User Interface

8.10 User Experience

The design focuses on a single task with one input box and one primary button, guides the student through placeholder text and shortcuts, distinguishes answered and unanswered questions with green and yellow messages, shows the category, confidence and matched FAQ with every answer, and turns failures into readable messages. No usability study or survey was conducted, so this describes the design rather than measured satisfaction.

CHAPTER 9 API AND BACKEND

9.1 Backend Overview

The backend (`backend/main.py`, `backend/database.py`, `hybrid_engine.py`) receives questions, runs the engine, stores history and returns JSON at `http://127.0.0.1:8000`.

9.2 FastAPI

FastAPI provides automatic request validation, JSON conversion and interactive documentation. The `/ask` body is described by the Pydantic model `QuestionRequest`, and the title, description and version from `main.py` appear in the generated OpenAPI description.

9.3 API Endpoint

Method	Path	Handler	Response Fields
GET	/health	health_check()	status ("healthy"), service
POST	/ask	ask_question()	found, question, answer, category, confidence
GET	/faqs	get_faqs()	count (34), faqs: id, category, question, answer
GET	/categories	get_categories()	count (11), categories (sorted)
GET	/history	history()	count, history records (newest first)

Table 9.1: API Endpoints and Responses

9.4 Request Structure

`/ask` expects a JSON body with one required string field, `question` (Code Snippet 9.1).

```
curl -X 'POST' 'http://127.0.0.1:8000/ask' \
  -H 'accept: */*' -H 'Content-Type: application/json' \
  -d '{ "question": "What is the weather today?" }'
```

Code Snippet 9.1: Example /ask Request Generated by the Swagger Interface

9.5 Response Structure

Field	Type	Description
found	boolean	true if the best score is at least 0.50
question	string / null	Matched FAQ question, or null
answer	string	Stored answer, or the fallback message
category	string	Matched category, or "Unknown"
confidence	number	Best final score between 0 and 1

Table 9.2: Response Fields of POST /ask

Code Snippet 9.2 shows the actual response for "What is the weather today?" captured in Swagger (Figure 9.2); 0.1129322424530983 corresponds to 11.29%. An answered question has the same structure with `found` = true; for "Where do I get books?" it contains "Where can I get books from the library?", category "Library" and confidence 0.8345 (rounded, as stored in the database).

```
{
  "found": false,
  "question": null,
  "answer": "Sorry, I could not find a relevant answer. Please contact the concerned
college department.",
  "category": "Unknown",
  "confidence": 0.1129322424530983
}
```

Code Snippet 9.2: Actual /ask Response for an Unrelated Question

9.6 Swagger Documentation

FastAPI generates Swagger UI at `/docs` (Figure 9.1), showing the title "Smart College FAQ API", the badges 1.0.0 and OAS 3.1, and the endpoints with the expanded POST /ask request body and 200 response.

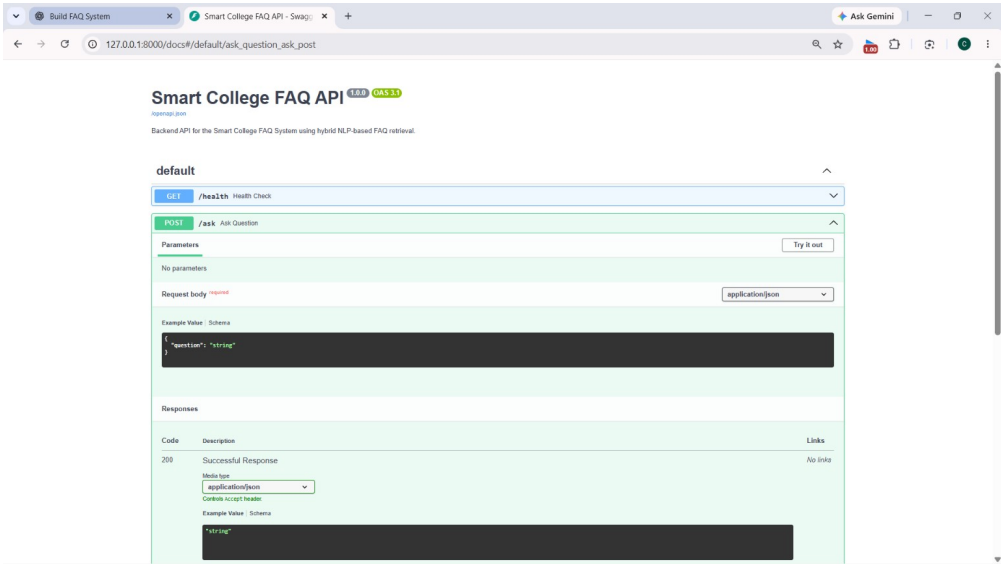


Figure 9.1: FastAPI Swagger Documentation Interface

Figure 9.2 shows `/ask` executed through "Try it out", with the curl command, response code 200, body and headers (`server: uvicorn`).

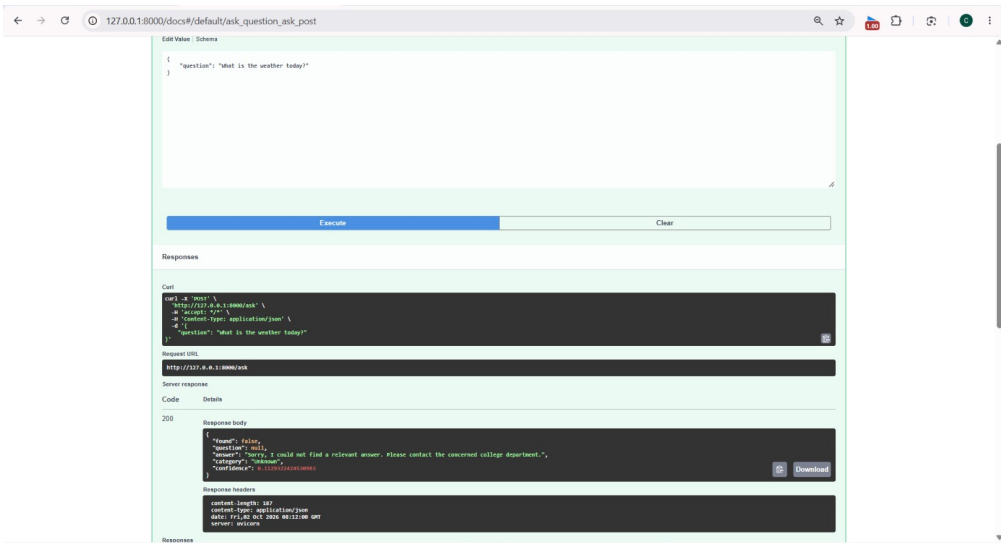


Figure 9.2: API Response for an Unknown Question in Swagger UI

9.7 Backend Processing Flow

Figure 9.3 shows the interactions for one question, from the frontend request through validation, scoring and saving to the displayed result. Model loading happens at start-up (Figure 4.8), and response times were not measured.

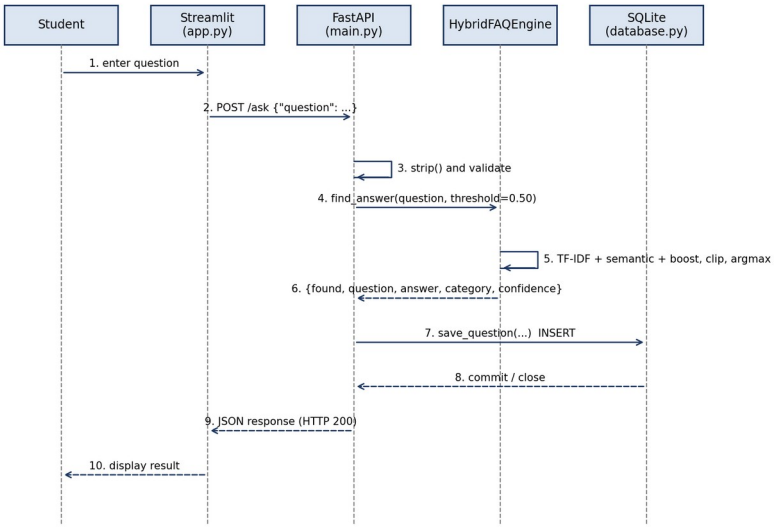


Figure 9.3: Sequence Diagram for Processing a Question Through POST /ask

9.8 Database/History

Each processed question is stored in `question_history` (Figure 4.10); created_at uses SQLite's CURRENT_TIMESTAMP (UTC). The supplied database holds 42 records from 1–2 October 2026, including two complete runs of the 14 validation questions (Appendix E).

9.9 Error Handling

Invalid bodies are rejected by FastAPI validation, empty questions are handled explicitly, unrelated questions are returned as a valid fallback result with HTTP 200, and each database function opens and closes its own connection. The backend does not define custom exception handlers for unexpected engine or database errors, which FastAPI would return as server errors; structured error handling and logging are listed in Chapter 12.

CHAPTER 10 TESTING AND VALIDATION

10.1 Testing Strategy

Testing was carried out at four levels, from the smallest component to the complete API, each supported by a script run from the command line (Table 10.1). The first four scripts import project modules directly; `test_api.py` sends real HTTP requests and requires the backend to be running. Each layer was verified before being relied on by the next.

Script	Level	What It Tests
test_preprocessing.py	Component	preprocess_text() output for four questions
test_engine.py	Retrieval (lexical)	FAQEngine best match, score and top three for seven questions
test_semantic.py	Retrieval (semantic)	Top three semantic matches for five questions
test_hybrid.py	Retrieval (hybrid)	HybridFAQEngine results for 22 questions
test_api.py	System (API)	Category from POST /ask for 14 questions vs. expected category

Table 10.1: Test Scripts and Their Purpose

10.2 Unit Testing

Unit-level testing is performed through component scripts that run one module with fixed inputs and print the outputs for inspection, rather than through a framework such as pytest. Only `test_api.py` compares results with expected values automatically and computes accuracy.

10.3 Preprocessing Testing

`test_preprocessing.py` produced the outputs listed in Table 6.3: "apply examination", "find college library", "working hour college" and "student register placement". Each output contains only lowercase content words with plural nouns singularised, confirming that all five preprocessing steps work as intended.

10.4 FAQ Retrieval Testing

`test_engine.py` runs the TF-IDF `FAQEngine` (threshold 0.20). Because TF-IDF scoring is deterministic for a fixed dataset, Table 10.2 gives the script's output on the project dataset.

Question	Best Match	Category	Score	Found
How do I apply for hostel?	Where can I apply for hostel accommodation?	Hostel	77.89%	Yes
Where can I see examination information?	How can I apply for an examination?	Examination	48.14%	Yes

Question	Best Match	Category	Score	Found
I want to register for placement	How can I register for placements?	Placement	100.00%	Yes
How do I borrow a library book?	How can I borrow a book from the library?	Library	100.00%	Yes
What is the attendance requirement?	What is the attendance requirement?	Attendance	100.00%	Yes
How can I apply for scholarship?	How can I apply for a scholarship?	Scholarship	100.00%	Yes
What is the weather today?	None	Unknown	0.00%	No

Table 10.2: TF-IDF Engine Test Results (test_engine.py)

All six college questions received an FAQ from the expected category and the weather question was rejected. However, "Where can I see examination information?" was matched to "How can I apply for an examination?" only through the shared word "examination" (48.14%), which does not match the intent of finding information. This illustrates the weakness of purely lexical matching.

10.5 Semantic Testing

`test_semantic.py` encodes the FAQ questions with all-MiniLM-L6-v2 and prints the top three semantic matches for five questions, including the paraphrase "Where can I see examination information?" and the unrelated "What is the weather today?". Its terminal output was not included in the material provided for this report, so individual semantic scores are not reproduced; the semantic contribution is evaluated through the hybrid and API results.

10.6 Hybrid Testing

`test_hybrid.py` runs the hybrid engine (threshold 0.50) on 22 questions: examination, library, hostel and placement questions (three each), fee, scholarship and attendance questions (two each), and four unrelated questions ("What is the weather today?", "Who is the Prime Minister of India?", "How do I cook rice?", "Tell me a joke."). It prints the matched FAQ, category, hybrid score, status and answer for inspection. Its output was not included in the provided material, so no scores from it are reported; quantitative results come from the API test.

10.7 API Testing

`test_api.py` defines 14 cases, each a question with an expected category. It posts each question to `/ask`, marks PASS if the returned category equals the expected one, and prints totals and accuracy; a request that raises an exception is counted as FAIL. The cases are eleven positive questions (one per category) and three negative questions expected to return "Unknown", which pass only if the threshold rejects them.

The results below were obtained with the backend running. The same 14 questions, with identical categories and confidence values, are recorded in the history database in two complete runs (2026-10-01 18:55:03 and 2026-10-02 08:27:35 UTC; Appendix E). A run made while the backend was not running (Figure F.1) failed with connection errors and is not part of these results.

10.8 Positive Test Cases

No.	Question	Expected	Actual	Confidence	Status
1	What is the admission process?	Admission	Admission	100.00%	PASS
2	Where can I find exam notices?	Examination	Examination	96.68%	PASS
3	How can I pay my college fees?	Fees	Fees	100.00%	PASS
4	Where do I get books?	Library	Library	83.45%	PASS
5	How can I get a hostel room?	Hostel	Hostel	100.00%	PASS
6	How much attendance do I need?	Attendance	Attendance	82.33%	PASS
7	I want to register for campus placement	Placement	Placement	90.75%	PASS
8	How can I get a scholarship?	Scholarship	Scholarship	88.50%	PASS
9	How can I contact my department?	Academic	Academic	100.00%	PASS
10	How can I join a college club?	Student Activities	Student Activities	100.00%	PASS
11	What are the college working hours?	General	General	100.00%	PASS

Table 10.3: Positive API Test Cases

Cases 1, 3, 5, 9, 10 and 11 are worded exactly as dataset questions and all scored 100.00%. The other five are paraphrases and scored between 82.33% and 96.68%, well above the threshold.

10.9 Negative Test Cases

No.	Question	Expected	Actual	Confidence	Status
12	What is the weather today?	Unknown	Unknown	11.29%	PASS
13	How do I cook rice?	Unknown	Unknown	15.85%	PASS
14	Tell me a joke.	Unknown	Unknown	10.33%	PASS

Table 10.4: Negative API Test Cases

10.10 Unknown Question Testing

Besides the API test, "What is the weather today?" was submitted manually through the Streamlit interface (Figure 8.5) and Swagger (Figure 9.2), and a variant with a trailing character ("What is the weather today? -") was rejected with 9.68%. None of the content words of the unrelated questions ("weather", "today", "cook", "rice", "tell", "joke") appear in the FAQ vocabulary, so their TF-IDF contribution is zero and their confidence comes only from 0.70 times the semantic similarity.

10.11 Test Results

Measure	Value
Total / positive / negative cases	14 / 11 / 3
Passed / failed	14 / 0
Accuracy on the 14-case validation set	100.00%
Confidence range, positive cases	82.33% to 100.00%
Confidence range, negative cases	10.33% to 15.85%

Table 10.5: API Validation Test Summary

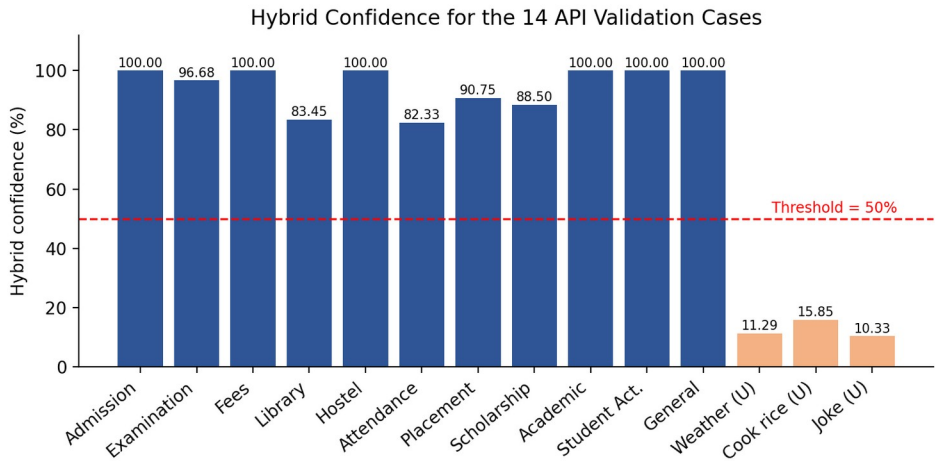


Figure 10.1: Hybrid Confidence for the 14 API Validation Cases

10.12 Accuracy Calculation

$$Accuracy = (Passed\ test\ cases / Total\ test\ cases) \times 100 = (14 / 14) \times 100 = 100.00\% \quad (10.1)$$

The system achieved 100% accuracy on the project's 14-case validation set. The set is small, has one positive question per category, contains six questions worded exactly as in the dataset, was written by the project team, and checks only the category rather than the specific FAQ record. The result shows that the system behaves as intended on these cases; it does not establish accuracy on all questions real students might ask.

CHAPTER 11 RESULTS AND DISCUSSION

11.1 System Results

The system works end to end: the screenshots show the interface answering library, hostel and placement questions and rejecting an unrelated question, and the backend responding through Swagger. The API test produced the expected category for all 14 cases, and the history database holds 42 records from the interface, Swagger and test runs, confirming that logging works.

11.2 FAQ Retrieval Performance

Retrieval performance is measured by category correctness and confidence on the validation set; speed and resource use were not measured. To isolate the semantic contribution, the TF-IDF baseline was run on the same 14 questions (Table 11.1, Figure 11.1).

Question	TF-IDF Baseline	Hybrid	Category
What is the admission process?	100.00%	100.00%	Same
Where can I find exam notices?	83.90%	96.68%	Same
How can I pay my college fees?	100.00%	100.00%	Same
Where do I get books?	81.65%	83.45%	Same
How can I get a hostel room?	100.00%	100.00%	Same
How much attendance do I need?	70.71%	82.33%	Same
I want to register for campus placement	100.00%	90.75%	Same
How can I get a scholarship?	59.45%	88.50%	Same
How can I contact my department?	100.00%	100.00%	Same
How can I join a college club?	100.00%	100.00%	Same
What are the college working hours?	100.00%	100.00%	Same
What is the weather today?	0.00%	11.29%	Both rejected
How do I cook rice?	0.00%	15.85%	Both rejected
Tell me a joke.	0.00%	10.33%	Both rejected

Table 11.1: TF-IDF Baseline Score and Hybrid Confidence for the Validation Questions

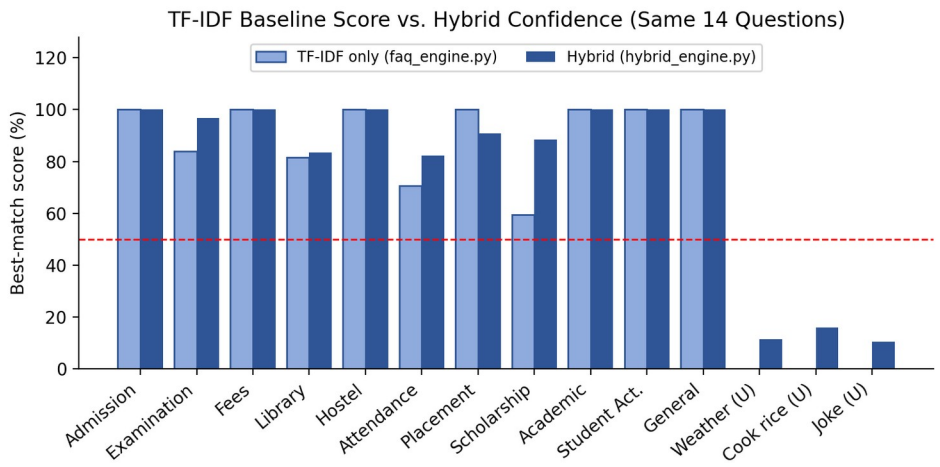


Figure 11.1: TF-IDF Baseline Score Compared with Hybrid Confidence

Both engines give the expected category on this set. The difference is in the strength of evidence: for paraphrases the hybrid score is generally higher ("How can I get a scholarship?" 59.45% to 88.50%; "How much attendance do I need?" 70.71% to 82.33%), giving a wider margin above the threshold. The two engines use different thresholds (0.20 and 0.50), so their scores are not directly comparable as decisions.

11.3 Semantic Matching Results

The clearest lexical failure is "Where can I see examination information?", for which TF-IDF chose "How can I apply for an examination?" through the single shared word "examination" (Table 10.2). Sentence embeddings represent the whole meaning of a question and are intended to address this weakness. Since the `test_semantic.py` scores were not available, the semantic similarity is recovered here from the hybrid scores using Equations 6.6 to 6.8, for cases where the matched FAQ and its TF-IDF score are known (Table 11.2).

Question	Matched FAQ	TF-IDF (T)	Boost (B)	Hybrid (C)	Implied S = (C − 0.30T − B) / 0.70
Where do I get books?	Where can I get books from the library?	0.8165	0.00	0.8345	≈ 0.842
I want to register for campus placement	How can I register for placements?	1.0000	0.05	0.9075	≈ 0.796
What is the weather today?	(rejected)	0.0000	0.00	0.1129	≈ 0.161 (highest)

Table 11.2: Decomposition of Hybrid Scores

For the library question the semantic similarity (about 0.84) exceeds the TF-IDF score, raising the hybrid score. For the placement statement TF-IDF is perfect, but the statement form and the word "campus" lower the semantic similarity (about 0.80), so the hybrid score falls below the TF-IDF

score. The weather question's highest semantic similarity with any FAQ is only about 0.16. These values are calculated from reported results and the code, not separately measured.

11.4 Hybrid Matching

Because the semantic weight is 0.70, a question with no lexical overlap can still pass the threshold if its semantic similarity is about 0.71 or more, while TF-IDF alone can contribute at most 0.30 (0.35 with the boost). The design therefore depends primarily on meaning, with lexical evidence as support. Clipping makes exact matches easy to recognise (all six exactly worded questions scored 100.00%), but it prevents the confidence from distinguishing degrees of near-exact matching.

11.5 Unknown Query Detection

All unrelated questions tested scored between 9.68% and 15.85%, while the lowest related question scored 82.33%, a gap of over 66 percentage points with the 50% threshold well inside it. These were clearly unrelated questions; partially related questions not covered by the dataset (for example, about the canteen or transport) were not tested and would likely receive intermediate scores, the hardest case for any threshold.

11.6 API Performance

The `/ask` endpoint returned correctly structured responses with HTTP 200 for every validation question and stored each result. In each complete validation run in the database, all 14 records were inserted within about one second, showing that the sequence of requests was handled without errors. Response time, throughput and concurrent load were not formally measured.

11.7 UI Evaluation

The interface displayed every element defined in the code, and answered and unanswered questions were visually distinct. No usability test or survey was conducted, so this evaluation is qualitative.

11.8 Discussion

The results support the project's main idea: combining lexical and semantic similarity with a confidence threshold allows common college questions in different wordings to be answered while unrelated questions are rejected, using a pretrained model without any training. The 100% accuracy applies only to the 14-case set; the TF-IDF baseline also matched all categories on this set, so the observed benefit of the hybrid engine lies in confidence margins and in examples such as the examination information question; and the parameters were set manually rather than optimised.

11.9 Observed Limitations

- Coverage is limited to 34 questions, and partially related questions were not tested.
- The API test checks only the category, not the specific FAQ record.
- Confidence is clipped at 100%.

- Two hostel records ("How can I apply..." and "Where can I apply for hostel accommodation?") have identical preprocessed text, so TF-IDF cannot distinguish them.
- Outputs of `test_semantic.py` and `test_hybrid.py` were not available for analysis, and response time was not measured.

CHAPTER 12 LIMITATIONS AND FUTURE SCOPE

12.1 Current Limitations

The system meets the objectives of this mini project but has limitations that must be addressed before use with real students. Table 12.1 summarises them.

Area	Limitation
Dataset	34 sample answers; uneven category coverage (General has one record)
NLP	Manual weights, boost and threshold; English only; no spelling correction; no conversation context
Retrieval	Single best answer; confidence clipped at 100%
Architecture	Local deployment at 127.0.0.1; no authentication; no admin interface
Code	Intent boost repeats preprocessing; no custom exception handling or logging
Testing	14-case set; category-level checks; no automated unit tests; no performance measurement

Table 12.1: Summary of Current Limitations

12.2 Dataset Limitations

The 34 records demonstrate the system but cover only part of what students ask, and some categories have very few records. The answers are general sample answers and must be replaced with verified, regularly updated institutional information before practical use. Near-duplicate records add little information for the TF-IDF component.

12.3 NLP Limitations

The weights, boost and threshold were not optimised on labelled data. The stopword list, lemmatizer and model are used for English only. Misspelled words do not match the TF-IDF vocabulary. The lemmatizer is called without part-of-speech information, so verbs are not reduced. Each question is handled independently, and the 18 intent words are hard-coded.

12.4 Dependency on FAQ Dataset

The system can answer only what the dataset contains and returns stored answers exactly as written, even if outdated. This also ensures that it never produces unreviewed information, but its usefulness is bounded by the dataset's quality, coverage and currency. Partially related questions are most at risk of receiving a loosely related answer.

12.5 Possible Improvements

Within the current architecture, the intent boost could reuse the stored `processed_question` values; the test scripts could become automated pytest tests; a larger, independently written validation set could be used to tune parameters and to check the matched FAQ rather than only the

category; the top three candidates could support "Did you mean ...?" suggestions; and backend logging, exception handlers and response-time measurement could be added.

12.6 Larger FAQ Dataset

Reviewing history records with category "Unknown" or low confidence would reveal missing topics. Since the engine rebuilds its representations from the CSV file at start-up, the dataset can be extended without code changes; for a much larger dataset, FAQ embeddings could be precomputed and stored.

12.7 Multilingual Support

Not implemented. Supporting Marathi, Hindi and other languages would require a multilingual embedding model, suitable preprocessing and answers in each language.

12.8 Voice-Based Queries

Not implemented. Speech-to-text in the frontend could feed recognised text to the existing `/ask` endpoint, and text-to-speech could read answers aloud.

12.9 Admin FAQ Management

Not implemented; the CSV file is edited directly and the backend restarted. An administrator interface could add, edit and delete FAQs and review unanswered questions, which would require storing FAQs in the database, management endpoints and refreshing the engine after changes.

12.10 Improved Ranking

Possible directions include parameter tuning on labelled data, re-ranking top candidates with a cross-encoder, fine-tuning the embedding model on college question pairs, and matching against FAQ answers as well as questions.

12.11 Authentication

Not implemented; all endpoints, including `/history`, are open to any client that can reach the server. Network deployment would require login and access control for students and administrators.

12.12 Cloud Deployment

Not implemented. Deployment on a college or cloud server would require a configurable backend address, a production server configuration, a database suited to concurrent use if needed, and planning for the model's memory requirements.

12.13 Advanced Conversational Capabilities

The system could remember context and handle follow-up questions while still answering only from verified information.

Future Enhancement	Description
Dataset and tuning	Expand coverage; optimise weights and threshold
Multilingual and voice	Indian languages; speech input and output
Admin and ranking	FAQ management interface; re-ranking and fine-tuning
Security and deployment	Authentication; college or cloud hosting; conversational features

Table 12.2: Summary of Future Scope

CHAPTER 13 CONCLUSION

13.1 Summary of the Work

This project addressed the problem of answering routine college questions that students ask repeatedly, which static FAQ pages, keyword search and office enquiries handle poorly when the student's wording differs from the stored text. The Smart College FAQ System was designed and implemented as an NLP-based retrieval system that accepts a natural-language question and returns the most relevant answer from a predefined FAQ dataset of 34 records in 11 categories.

13.2 NLP Approach and Hybrid Retrieval

Questions are preprocessed (lowercasing, special character removal, tokenization, stopword removal and lemmatization) and represented with TF-IDF, while the original text is encoded with the pretrained model all-MiniLM-L6-v2. The two cosine similarities are combined with weights of 0.30 and 0.70, an intent boost of 0.05 is added and the result is clipped to 0–1. A threshold of 0.50 decides between returning the best FAQ and a fallback message, allowing paraphrased questions to be matched while unrelated questions are rejected.

13.3 Backend, Frontend and Data Storage

A FastAPI backend exposes the engine through `/health`, `/ask`, `/faqs`, `/categories` and `/history`, with Pydantic validation and Swagger documentation, and stores every processed question in SQLite. A Streamlit frontend lets students type or select a question and shows the answer with its category, confidence and matched FAQ, or a clear warning.

13.4 Testing and Results

On the project's 14-case API validation set (eleven college questions and three unrelated questions), the system returned the expected category in all cases, giving 100% accuracy on this validation set. College questions scored 82.33% to 100.00% and unrelated questions 10.33% to 15.85%, and the hybrid engine gave stronger confidence than the TF-IDF baseline to several paraphrases. Because the set is small, these results do not represent accuracy on all student questions.

13.5 Practical Usefulness and Future Improvement

The project shows that a useful college FAQ assistant can be built from free, open-source components on an ordinary computer without training a model, while returning only pre-written answers and stating clearly when it cannot help. With a verified and expanded dataset, tuned parameters, and features such as multilingual input, FAQ administration, authentication and server deployment, it could reduce the repetitive load on college offices and give students a quick first reference for routine information.

REFERENCES

[1] Python Software Foundation, "Python 3.13 Documentation." [Online]. Available: https://docs.python.org/3.13/

[2] S. Bird, E. Klein and E. Loper, *Natural Language Processing with Python*. Sebastopol, CA: O'Reilly Media, 2009.

[3] NLTK Project, "Natural Language Toolkit Documentation." [Online]. Available: https://www.nltk.org/

[4] G. A. Miller, "WordNet: A Lexical Database for English," *Communications of the ACM*, vol. 38, no. 11, pp. 39–41, 1995.

[5] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," *Information Processing & Management*, vol. 24, no. 5, pp. 513–523, 1988.

[6] C. D. Manning, P. Raghavan and H. Schütze, *Introduction to Information Retrieval*. Cambridge, UK: Cambridge University Press, 2008.

[7] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[8] scikit-learn developers, "TfidfVectorizer – scikit-learn documentation." [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html

[9] A. Vaswani et al., "Attention Is All You Need," in *Advances in Neural Information Processing Systems 30 (NIPS 2017)*, 2017.

[10] J. Devlin, M.-W. Chang, K. Lee and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in *Proceedings of NAACL-HLT 2019*, pp. 4171–4186, 2019.

[11] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in *Proceedings of EMNLP-IJCNLP 2019*, pp. 3982–3992, 2019.

[12] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang and M. Zhou, "MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers," in *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, 2020.

[13] Hugging Face, "sentence-transformers/all-MiniLM-L6-v2 – Model Card." [Online]. Available: https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2

[14] Sentence Transformers, "SentenceTransformers Documentation." [Online]. Available: https://www.sbert.net/

[15] W. McKinney, "Data Structures for Statistical Computing in Python," in *Proceedings of the 9th Python in Science Conference*, pp. 56–61, 2010.

[16] FastAPI, "FastAPI Documentation." [Online]. Available: https://fastapi.tiangolo.com/

[17] Pydantic, "Pydantic Documentation." [Online]. Available: https://docs.pydantic.dev/

[18] Uvicorn, "Uvicorn Documentation." [Online]. Available: https://www.uvicorn.org/

[19] Streamlit Inc., "Streamlit Documentation." [Online]. Available: https://docs.streamlit.io/

[20] SQLite Consortium, "SQLite Documentation." [Online]. Available: https://www.sqlite.org/docs.html

[21] Python Software Foundation, "sqlite3 – DB-API 2.0 interface for SQLite databases." [Online]. Available: https://docs.python.org/3/library/sqlite3.html

[22] K. Reitz et al., "Requests: HTTP for Humans – Documentation." [Online]. Available: https://requests.readthedocs.io/

[23] OpenAPI Initiative, "OpenAPI Specification v3.1.0." [Online]. Available: https://spec.openapis.org/oas/v3.1.0

APPENDIX A PROJECT STRUCTURE

The structure below is taken from the VS Code Explorer view (Figure F.1); folder contents not shown there are not expanded.

```
Smart-College-FAQ/
|-- backend/
|   |-- __init__.py
|   |-- database.py
|   |-- faq_history.db
|   `-- main.py
|-- data/
|   `-- faq_data.csv
|-- models/
|-- screenshots/
|-- venv/
|-- app.py
|-- faq_engine.py
|-- hybrid_engine.py
|-- preprocessing.py
|-- reqtorun
|-- requirements.txt
|-- test_api.py
|-- test_engine.py
|-- test_hybrid.py
|-- test_preprocessing.py
|-- test_semantic.py
`-- Smart-College-FAQ.zip
```

Code Snippet A.1: Directory Structure of the Smart-College-FAQ Project

The system is started in two terminals from the project root directory: the FastAPI backend is started with Uvicorn using the module path `backend.main:app` (which serves the API at `http://127.0.0.1:8000`), and the Streamlit frontend is started with `streamlit run app.py`. The API validation is run with `python test_api.py` while the backend is running.

APPENDIX B DATASET SAMPLE

Table B.1 lists the id, category and question of all 34 records of `data/faq_data.csv`. Each record also has an answer field; the answers are general sample answers such as "Students can check the official scholarship notices and information provided by the college." and are not verified official policies.

ID	Category	Question
1	Admission	What is the admission process?
2	Admission	How can I take admission in the college?
3	Admission	What documents are required for admission?
4	Examination	How can I apply for an examination?
5	Examination	Where can I find examination notices?
6	Examination	When are the semester examinations?
7	Examination	Where can I check the examination schedule?
8	Fees	How can I pay my college fees?
9	Fees	Where can I get information about college fees?
10	Library	What are the library timings?
11	Library	How can I issue a book from the library?
12	Library	How many books can a student borrow?
13	Hostel	How can I apply for hostel accommodation?
14	Hostel	What is the hostel admission process?
15	Hostel	What are the hostel facilities?
16	Attendance	What is the attendance requirement?
17	Attendance	How can I check my attendance?
18	Placement	How can I register for placements?
19	Placement	Where can I get placement information?
20	Placement	What is the placement process?
21	Scholarship	How can I apply for a scholarship?
22	Scholarship	Where can I find scholarship information?
23	Academic	How can I contact my department?
24	Academic	Where can I find academic notices?
25	Academic	How can I contact a faculty member?
26	Student Activities	How can I participate in college events?
27	Student Activities	How can I join a college club?
28	General	What are the college working hours?
29	Library	Where can I get books from the library?
30	Library	How can I borrow a book from the library?
31	Library	Where can I find books in the library?
32	Hostel	How can I get hostel accommodation?
33	Hostel	How can I get a hostel room?
34	Hostel	Where can I apply for hostel accommodation?

Table B.1: FAQ Questions in data/faq_data.csv

APPENDIX C IMPORTANT SOURCE CODE

This appendix reproduces the preprocessing module, which is shared by both engines; the other modules are excerpted in Chapter 7. Blank lines have been removed to save space; the code is otherwise unchanged.

C.1 preprocessing.py

Text preprocessing module used by both engines.

```
import re
import nltk
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
# Download required NLTK resources
nltk.download("stopwords")
nltk.download("wordnet")
nltk.download("omw-1.4")
# Load English stopwords
stop_words = set(stopwords.words("english"))
# Create lemmatizer
lemmatizer = WordNetLemmatizer()

def preprocess_text(text):
    """
    Preprocess text for the FAQ NLP system.
    Steps:
    1. Convert text to lowercase
    2. Remove special characters and numbers
    3. Tokenize text
    4. Remove stopwords
    5. Lemmatize words
    6. Return cleaned text
    """
    # Convert to lowercase
    text = text.lower()
    # Remove special characters and numbers
    text = re.sub(r"[^a-zA-Z\s]", " ", text)
    # Tokenization
    words = text.split()
    # Remove stopwords and perform lemmatization
    processed_words = []
    for word in words:
        if word not in stop_words:
            lemma = lemmatizer.lemmatize(word)
            processed_words.append(lemma)
    # Convert list back into a sentence
    return " ".join(processed_words)
```

Code Snippet C.1: preprocessing.py

APPENDIX D API EXAMPLES

The responses below follow directly from `backend/main.py` and the current dataset. Actual /ask responses are shown in Code Snippet 9.2 and Figure 9.2.

```
GET /health
{ "status": "healthy", "service": "Smart College FAQ API" }


GET /categories
{ "count": 11,
  "categories": ["Academic", "Admission", "Attendance", "Examination", "Fees", "General",
                 "Hostel", "Library", "Placement", "Scholarship", "Student Activities"] }


POST /ask   body: { "question": "   " }
{ "found": false, "answer": "Please enter a question.", "category": "Unknown",
"confidence": 0.0 }
```

Code Snippet D.1: Responses of GET /health, GET /categories and POST /ask with an Empty Question

APPENDIX E TEST RESULTS

Table E.1 lists the records stored in `backend/faq_history.db` during the second complete run of the 14 API validation questions. The first complete run (records 5 to 18, stored at 2026-10-01 18:55:03 UTC) contains the same questions with identical categories and confidence values. Timestamps are in UTC, as recorded by SQLite's CURRENT_TIMESTAMP.

ID	Question	Category	Confidence	Found	Created At (UTC)
29	What is the admission process?	Admission	1.0000	1	2026-10-02 08:27:35
30	Where can I find exam notices?	Examination	0.9668	1	2026-10-02 08:27:35
31	How can I pay my college fees?	Fees	1.0000	1	2026-10-02 08:27:35
32	Where do I get books?	Library	0.8345	1	2026-10-02 08:27:35
33	How can I get a hostel room?	Hostel	1.0000	1	2026-10-02 08:27:35
34	How much attendance do I need?	Attendance	0.8233	1	2026-10-02 08:27:35
35	I want to register for campus placement	Placement	0.9075	1	2026-10-02 08:27:36
36	How can I get a scholarship?	Scholarship	0.8850	1	2026-10-02 08:27:36
37	How can I contact my department?	Academic	1.0000	1	2026-10-02 08:27:36
38	How can I join a college club?	Student Activities	1.0000	1	2026-10-02 08:27:36
39	What are the college working hours?	General	1.0000	1	2026-10-02 08:27:36
40	What is the weather today?	Unknown	0.1129	0	2026-10-02 08:27:36
41	How do I cook rice?	Unknown	0.1585	0	2026-10-02 08:27:36
42	Tell me a joke.	Unknown	0.1033	0	2026-10-02 08:27:36

Table E.1: Stored Records of the API Validation Run (Records 29 to 42)

Summary: Total Tests 14, Passed 14, Failed 0, Accuracy 100.00% on the 14-case validation set. The supplied database contains 42 records in total: 31 records with `found` equal to 1 and 11 records with `found` equal to 0. The records cover the two complete validation runs (28 records) and 14 additional questions submitted individually through the user interface and the Swagger interface while the system was being demonstrated.

APPENDIX F ADDITIONAL SCREENSHOTS

Figure F.1 shows the VS Code development environment with the project structure in the Explorer panel and the integrated terminal. In this run, `test_api.py` was executed while the FastAPI backend was not running. Each request failed with a connection error ("No connection could be made because the target machine actively refused it"), and the script reported each affected test as FAIL with the error message. The screenshot demonstrates the exception handling of the test script and the requirement that the backend must be running before the API test is executed; it is not part of the validation results reported in Chapter 10, which were obtained with the backend running.

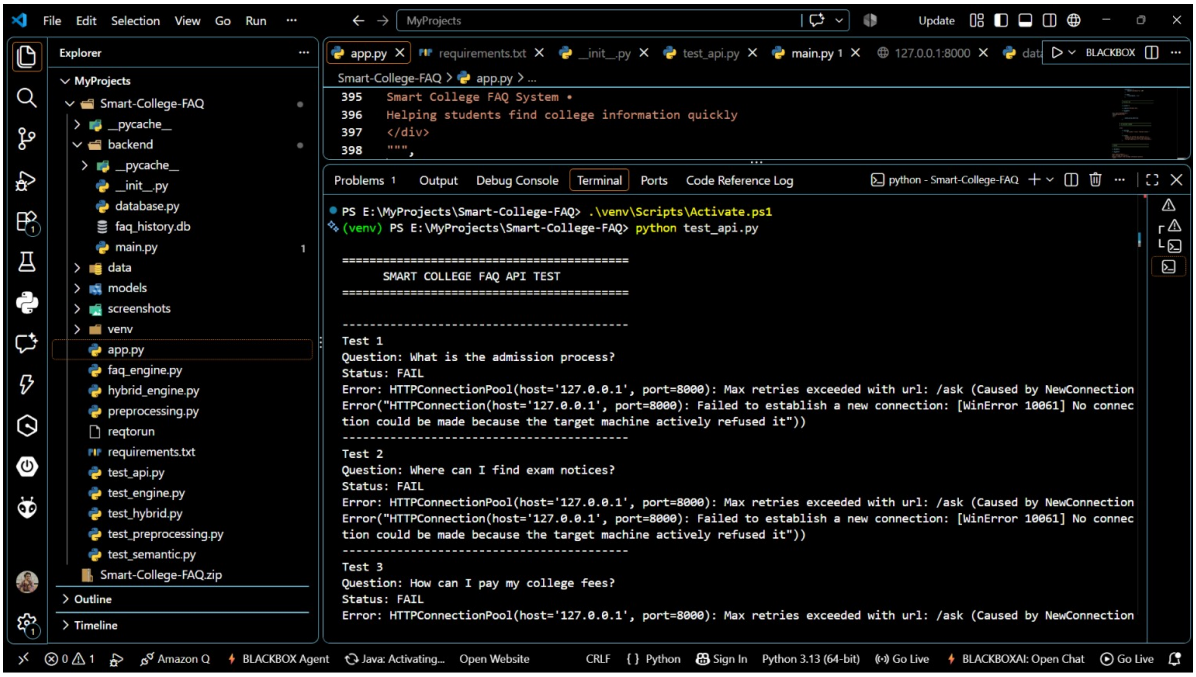


Figure F.1: test_api.py Executed While the Backend Was Not Running (VS Code)